

UNIVERSITÉ LIBRE DE BRUXELLES

Faculté des Sciences

Département d'Informatique

An Experimental Framework for Evaluating Traffic Deviation Plans

CALLENS HUGO

Promotors :

PROF. GIANLUCA BONTEMPI

ELADIO MONTERO PORRAS

DAVIDE ANDREA GUASTELLA

Master Thesis in Computer Sciences

Academic year 2025 – 2026

Abstract

Roadworks and scheduled maintenance force city authorities to close road segments and publish deviation plans, yet the choice between candidate detours is usually made by expert judgement and inspected only qualitatively. This thesis develops a reproducible experimental framework that turns that choice into a measurable comparison. From a SUMO road network and a Traffic Analysis Zone layer, the framework synthesises network-aware demand using a gravity origin-destination model, generates candidate detours around a closed edge (manually, with Yen’s k -shortest loopless paths, and with an upstream search that intercepts traffic before it reaches the closure), rewrites the routes of affected vehicles, and evaluates every candidate against a paired baseline over repeated Monte Carlo simulations with randomised route assignment. A decision layer rejects infeasible plans, screens the rest for Pareto dominance and ranks the survivors by additive weighted scoring, using Dirichlet weight sampling to test how far the ranking depends on the chosen weights.

The framework is exercised on three scenarios of increasing realism: a synthetic `city_tiny` grid, an OpenStreetMap topology around Flagey in Brussels and a closure of the Delta tunnel in Auderghem under both night-time and all-day regimes. On the synthetic network, automated generation widened the candidate set beyond the manually defined corridors, surfacing both shorter replacement paths and interception points upstream of the closure itself. On both real topologies the plan minimising aggregate delay was not the one distributing the burden most evenly, so the recommendation held in only about three quarters of sampled weight vectors (a trade-off the decision layer exposes rather than hides behind a single number). Re-scoring the Delta candidates under full-day exposure left the recommended plan unchanged but widened the worst-to-best delay ratio from 2.6 to 9.6, showing that candidates must be evaluated at the demand they will actually experience. All demand is synthetic, so the absolute figures are not calibrated predictions for any real city.

Contents

Abstract	i
1 Introduction	1
1.1 Digital Twin	1
1.2 Context	2
1.3 Objective	3
2 Foundations and Related Work	4
2.1 Traffic digital twins for decision support	4
2.1.1 Digital twins in traffic simulation	4
2.1.2 Simulation and mobility	5
2.1.3 Positioning of this work	5
2.2 Traffic simulation tools	6
2.2.1 Purpose of traffic simulation tools	6
2.2.2 Levels of detail in traffic simulation	6
2.2.3 SUMO as the selected microscopic simulator	7
2.2.4 Alternatives to SUMO	8
2.2.5 Choice of SUMO for this thesis	8
2.3 Stochastic simulation	9
2.3.1 Theoretical foundations of the Monte Carlo method	9
2.3.2 Randomness in microscopic traffic simulation	10
2.3.3 Determining the number of simulation runs	11
2.4 Multiple-criteria decision analysis	11
2.4.1 Pareto dominance and non-dominated plan sets	11
2.4.2 Additive weighted scoring	12
2.4.3 Weight sensitivity and robustness analysis	12
3 Methodology	13
3.1 System overview and pipeline	13
3.1.1 Concepts and contribution boundary	13
3.2 Scenario generation	15
3.2.1 Module overview	15
3.2.2 Stage 1 : Demand generation	16
3.2.3 Stage 2 : Route library	20
3.2.4 Stage 3 : Trip Instantiation	21
3.3 Deviation plan computation	21
3.3.1 Junction-level graph representation	22
3.3.2 Detour endpoint computation	23
3.3.3 Candidate generation	24
3.3.4 Route rewriting	25
3.4 Monte Carlo Simulation	27
3.4.1 Experimental design	27
3.4.2 Per-run routing and detour application	28

3.4.3	Iteration structure	28
3.4.4	Performance metrics	30
3.4.5	Convergence and number of runs	30
3.5	Evaluation and decision support	31
3.5.1	Choice of evaluator metrics	31
3.5.2	Comparable run set	32
3.5.3	Paired network-level statistics	32
3.5.4	Feasibility gate	33
3.5.5	Impacted-vehicle metrics	33
3.5.6	Multi-criteria decision analysis	34
4	Results	36
4.1	Experimental setup	37
4.2	Manually-defined plans	38
4.2.1	Evaluator verdict	41
4.3	Yen-based plans	41
4.3.1	Evaluator verdict	43
4.4	Upstream-based plans	43
4.4.1	Evaluator verdict	45
4.5	Comparison across generation strategies	45
4.6	Real-neighbourhood validation	46
4.6.1	Evaluator verdict	48
4.7	Closure of the Delta tunnel	50
4.7.1	Experimental setup	50
4.7.2	Evaluator verdict	51
4.7.3	Night closure versus all-day closure	53
5	Discussion	56
5.1	Usefulness of the framework	56
5.2	Limitations	57
5.2.1	Single-closure assumption	57
5.2.2	No intra-zone demand	57
5.2.3	Hourly OD time resolution	58
5.2.4	Demand fixed across Monte Carlo runs	58
5.2.5	Single vehicle class	58
5.2.6	Static plan generation	58
6	Conclusion	59
	Disclosure of AI Tools	62

1 Introduction

1.1 Digital Twin

A **digital twin** is a data-driven virtual representation of real-world entities designed to mirror a physical object or system throughout its lifecycle. The virtual part is updated with real-time data gotten from sensors or other sources making it possible to link the physical and digital parts. [1, 2]

A digital twin consists of three components [3] :

1. **Physical entity** : The real-world object or system being modeled (e.g., traffic, a plant)
2. **Virtual representation** : A model that replicates the behavior of the physical entity.
3. **Connection** : The link between the physical system and the virtual system making it possible to update the physical system by exchanging information via this real-time connection.

Figure 1 summarises this relationship between the physical object, the virtual model and the data connection.

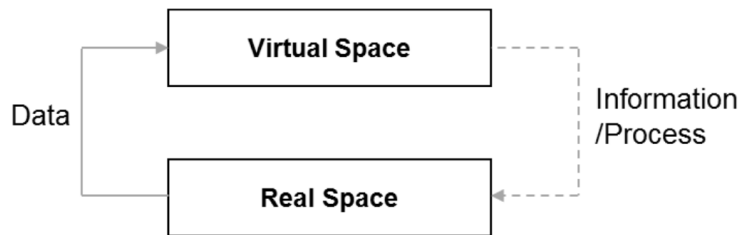


Figure 1: Illustration of a digital twin from [4]

Digital twins receive continuously streams of data from sensors or other sources ensuring that the virtual model replicates the current state of the physical object. The main goals for a digital twin are to simulate the physical entity and predict what could happen in order to optimize or predict failures. Urban traffic networks, which are both data-rich and fast-evolving, provide fertile ground for the deployment of digital twins. Figure 2 gives an example workflow for linking SUMO to map and sensor inputs in this context. Digital twins in traffic can potentially simulate and optimize traffic flows, supporting smarter city planning and real-time operations. A detailed discussion of modern digital twin solutions in traffic management is provided in **Section 2.1.1**.

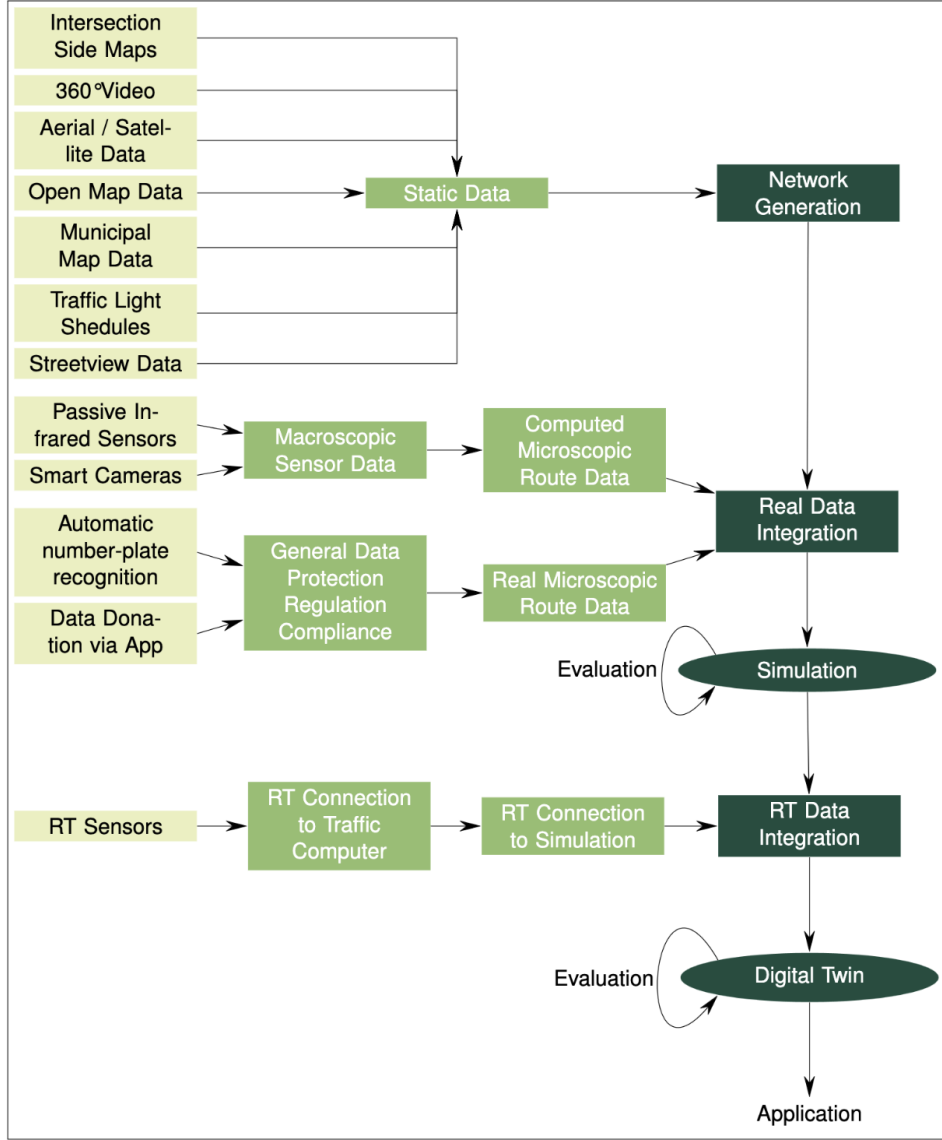


Figure 2: Example workflow for driving SUMO towards a digital twin. Various static and real-time data sources (maps, sensors, cameras) are integrated into simulation models to continuously update and calibrate the digital twin for real-world application. Source : [5]

1.2 Context

In this project, a digital twin acts as a precise and efficient method to simulate traffic based on real-world data. Digital twins leverage continuous data streams (for example, from vehicle counting devices) to create an up-to-date virtual model of physical traffic flows. Accurate traffic simulation relies on high-resolution data, typically gathered by a variety of counting devices such as inductive loop traffic detectors, Bluetooth recognition systems and ANPR (Automatic Number Plate Recognition) cameras [6]. These devices measure the number of vehicles passing specific locations over time, producing the data needed for dynamic modeling and simulation.

Through the aggregation and analysis of such data, it becomes possible to estimate traffic density and infer vehicle movement patterns across a road network. In this framework,

physical traffic corresponds to the physical entity of the digital twin, real-time vehicle counts represent the virtual layer and the counting devices themselves serve as the communication link between the physical and digital components.

Traffic simulation is particularly vital in urban environments such as Brussels, where the frequent congestion and ongoing roadworks demand dynamic management of the transport network. Planned deviations due to construction may alter typical traffic flows and produce unexpected bottlenecks. Accurate simulation enables city planners to evaluate alternative deviation plans and identify those that minimize congestion and disruption. Additionally, the direct and indirect effects of traffic interventions, such as road closures, can be promptly assessed.

1.3 Objective

The main objective of this work is to develop a reproducible experimental framework for evaluating deviation plans when a road segment is closed. Instead of focusing on the migration of an existing interface, this thesis studies how a SUMO-based traffic digital twin, stochastic routing and repeated simulation can be combined to compare candidate detours in a structured way. The work is divided into several sub-goals.

- **Scenario generation:** The construction of a synthetic, network-aware demand scenario from a SUMO road network and Traffic Analysis Zone data.
- **Deviation-plan computation:** The generation of candidate detour paths around a closed edge or sequence of edges, together with route-rewriting rules that apply these plans to affected vehicles.
- **Stochastic simulation:** The execution of repeated SUMO simulations using randomized routing seeds and paired baseline runs in order to capture variability in route choice and traffic outcomes.
- **Evaluation and decision support:** The comparison of candidate plans using traffic indicators, paired statistics, feasibility checks and multi-criteria decision analysis.
- **Framework validation:** The application of the full pipeline to controlled experiments and a Brussels neighbourhood case study in order to assess its usefulness and limitations.

Ultimately, the goal is to support city planners and traffic managers with a data-driven method for comparing deviation plans before they are deployed on the road network.

2 Foundations and Related Work

2.1 Traffic digital twins for decision support

2.1.1 Digital twins in traffic simulation

A traffic digital twin is useful here less as a real-time control system than as a planning environment in which a road manager can ask a concrete counterfactual question: *what happens if a road segment is closed and traffic is assigned to one of several deviation plans?* The literature on digital twins for mobility emphasizes this ability to reproduce a road network, ingest transport data, simulate alternative scenarios and support decision making before actions are deployed in the physical network [7, 8, 9]. This thesis uses that same decision-support idea, but it implements it as an offline experimental pipeline rather than as a continuously synchronized operational twin.

The framework developed in this thesis therefore separates the physical city from the experimental model used for plan comparison. The physical layer is represented by a SUMO road network and traffic analysis zones, the scenario layer generates synthetic, network-aware origin-destination demand, the planning layer converts a selected closure into candidate detour paths and the evaluation layer simulates and ranks those candidates. In this sense, the framework should be understood as an offline decision-support tool rather than as a fully operational traffic-management digital twin. It does not continuously ingest live sensor data or maintain a real-time estimate of the current traffic state. Its contribution lies instead in enabling the reproducible comparison of deviation policies under controlled assumptions.

The result is a decision-support digital twin prototype for roadwork planning. Its purpose is not to predict the exact state of Brussels traffic at a particular minute. Instead, it provides a repeatable environment for testing how different detour definitions change route assignment, vehicle time loss, travel time and the number of affected users. This framing is important because roadwork decisions are often made before the disruption occurs. The planner needs evidence about candidate plans, not only a live dashboard after congestion has already formed.

This scope follows from the exploratory work carried out before the present implementation. Several digital-twin approaches were considered: broad smart-city twins that integrate many urban systems, operational traffic twins that continuously assimilate sensor data, SUMO-based twins focused on traffic simulation and *TrafficTwin*, an earlier interactive prototype for deviation-plan assessment [10, 9, 5, 11]. *TrafficTwin* is the closest predecessor because it already frames a closure as a planning problem and lets a user inspect the impact of a deviation plan. The difference is that this thesis turns that idea into a reproducible experimental pipeline: demand generation, candidate-plan generation, stochastic re-routing, simulation, statistical comparison and multi-criteria ranking are explicit stages rather than manual or single-run interactions.

2.1.2 Simulation and mobility

Traffic simulation is the mechanism that turns the digital-twin idea into an experiment. The common theme across simulation-based traffic studies is that a proposed intervention can be tested in a controlled environment before it is implemented on the road. Prior work uses this principle for adaptive signal control, co-simulation and AI-based traffic management [12, 13, 14]. Those studies are useful background, but they usually use simulation to design or assess systems that adapt during operation, for example a controller that changes signal phases or an AI method that learns traffic-management actions. The intervention studied here is different. The closure and candidate detour policies are defined before the simulation runs and simulation is used to measure how each planned response affects delay, travel time and impacted users.

Another related line of work concerns real-time route recommendations from navigation applications such as Google Maps¹ and Waze². Scientific studies of online routing use these services as examples of systems that can redirect demand dynamically and, if many users react to similar information, can also create network-level effects that differ from a system optimum [15, 16]. This literature is relevant because it treats routing guidance as an intervention on traffic flow. The present thesis differs from it in both time scale and control objective: it does not provide live individualized advice to drivers, but evaluates planned deviation policies offline before a roadwork is deployed.

The simulation problem addressed here has three practical requirements. First, the same demand must be reused across alternatives so that candidate plans are compared on equal terms. Second, the experiment should test the sensitivity of plan rankings to plausible route-choice dispersion. Traffic assignment theory distinguishes deterministic shortest-path or user-equilibrium assignment from stochastic formulations in which travellers choose between alternatives according to perceived generalized costs rather than a single perfectly observed cost [17]. Third, the final recommendation must combine several outputs, since a plan that reduces one metric may worsen another. For example, a detour can reduce queueing and time loss near the closed edge by spreading traffic over alternative roads, while also increasing the mean travel time of affected vehicles because they are sent through a longer path.

In this setting, simulation is not used only for visualization. It is the measurement instrument of the thesis. The output of each SUMO run is parsed into quantitative indicators and the evaluator uses the distribution of those indicators over repeated runs to decide which deviation plan is most robust for the selected closure.

2.1.3 Positioning of this work

The problem addressed in this thesis is narrower than building a complete platform for traffic management. The practical question is: *if a road edge is closed, how can several candidate deviation plans be compared before the closure is deployed?* Existing work provides useful pieces of that answer. SUMO studies explain how to build reproducible simulation scenarios from a road network and demand data [18, 19]. Transport digital-twin

¹<https://www.google.com/maps/>

²<https://www.waze.com/>

research motivates the use of simulation models to test planning decisions before acting on the physical network [8, 9]. Online routing studies show that routing guidance can change network flows, although they usually study live advice to individual drivers rather than planned roadwork policies [15, 16]. At the tool level, *TrafficTwin* is the closest predecessor because it already treats a road closure as a deviation-plan assessment problem [11].

The contribution of this thesis is to connect these ideas into one reproducible evaluation pipeline. More specifically, it generates synthetic demand for a SUMO network, constructs candidate detours around a selected closure, rewrites affected routes, reruns the same scenario under stochastic routing assumptions and ranks the resulting plans with paired statistics and multi-criteria decision analysis. The emphasis is therefore not on a new simulator or a real-time routing service, but on a transparent workflow for comparing roadwork deviation policies under the same experimental conditions.

2.2 Traffic simulation tools

2.2.1 Purpose of traffic simulation tools

Traffic congestion is one of the major challenges faced by modern cities and is expected to get worse with increasing population and vehicle usage [20, 21]. Traffic simulation tools such as SUMO³, Vissim⁴, Aimsun⁵, or CityFlow⁶ are mathematical and computational models used to represent traffic dynamics, test network designs and estimate operational or environmental effects before an intervention is implemented [17, 22].

2.2.2 Levels of detail in traffic simulation

Traffic simulations are commonly categorized into three main types based on the level of detail desired [17, 22]:

1. **Microscopic Simulation:** Models each traffic participant (e.g., vehicles, pedestrians) individually. Agents continuously in time and space respond to their immediate environment. The global traffic dynamics is the result of the sum of these individual decisions and behaviors.
2. **Mesoscopic Simulation:** Still represents individual agents, but their behavior is determined by average traffic conditions like speed or density. Agents only interact directly at key points in the network, such as intersections.
3. **Macroscopic Simulation:** Looks at traffic as a whole rather than focusing on individual vehicles.

Choosing a simulation type entails balancing computational cost and realism. For instance, microscopic simulation is ideal for projects that require modeling of individual behaviors or the testing of interventions at the street or intersection level, which is often necessary for a dense urban environment such as Brussels. In contrast, a macroscopic simulation

³<https://sumo.dlr.de/>

⁴<https://www.ptvgroup.com/en/products/ptv-vissim/>

⁵<https://www.aimsun.com/>

⁶<https://cityflow-project.github.io/cityflow/>

might suffice for citywide traffic forecasting or strategic planning, where the focus is on broader patterns rather than local details.

2.2.3 SUMO as the selected microscopic simulator

SUMO (Simulation of Urban MObility) is an open-source, highly portable, microscopic and continuous multi-modal traffic simulation tool. It allows detailed modeling of individual vehicles, each following its own route through a specified road network. As a microscopic simulator, SUMO treats every vehicle as an individual entity with specific behaviors and interactions. The tool is multi-modal enabling the simulation of diverse traffic participants including cars, public transport vehicles and pedestrians [23] [24] [25].

SUMO has been used extensively to model real-world traffic systems in diverse urban contexts, including cities such as Berlin, Dublin and Turin [26]. Each scenario adapts to the specific traffic dynamics and urban characteristics of its location :

1. **Berlin:** Berlin has served for large-scale traffic simulations using SUMO. The "24 Hours of Traffic in Berlin" scenario captures over 2.2 million trips within an $800km^2$ area. This model enhances traditional traffic simulations by incorporating motorized private vehicles alongside realistic bus schedules derived from actual General Transit Feed Specification (GTFS) data, representing 89% of the city's bus lines. This scenario allows for holistic analyses that combine traffic, communication (including V2X and LTE/5G) and application simulations, making it suited for smart mobility researches. [27]
2. **Dublin:** The study by Gueriau and Dusparic [28] leverages SUMO to simulate and analyze the impact of connected and autonomous vehicles (CAVs) on real Irish road networks, covering urban, national and motorway settings. Using detailed demand and network data from Dublin and surrounding areas, the research quantifies how varying CAV penetration rates affect both traffic efficiency and open dataset for benchmarking the effects of emerging vehicle technologies in mixed traffic environments.
3. **Turin:** Leveraging real-world data, the TuSTScenario [29] provides a highly detailed SUMO-based simulation of metropolitan Turin, covering over $602.61km^2$. The model integrates structural road information from OpenStreetMap, validated average speeds from Google APIs, actual traffic light programming and real trip demands from local mobility services. With more than 2.2 million simulated daily trips and validated against city sensor data, this scenario enables realistic studies of urban traffic dynamics and congestion patterns.

Brussels is also a large city warranting similarly large-scale simulations. However, the focus of this study is distinct: rather than prioritizing multi-modal integration or CAV impact, we specifically investigate the planning and management of roadworks within the Brussels network. The goal is to demonstrate how such interventions can be realistically modeled, analyzed and optimized in SUMO for modern urban infrastructure management.

SUMO has been used to evaluate the effects of technologies like virtual traffic lights communication systems where cars directly communicate with each other. [30] In this study, Tonguz found that virtual traffic lights could reduce congestion and the likelihood of traffic accidents. Moreover SUMO can be used in environmental research, for instance

in assessing the impact of traffic-related emissions on urban air quality. In such cases traffic flows are generated from sensor data to estimate real-world emission patterns [31].

SUMO runs can be made reproducible when the same version, input files, command-line options and random seed are used. This is useful for controlled experiments because differences between scenarios can be traced to the tested intervention rather than to uncontrolled simulation changes. However, one fixed run only represents one possible realization of traffic behavior. Capturing variability in route choice and driving behavior therefore requires stochastic mechanisms and repeated simulations with different seeds [22, 32].

2.2.4 Alternatives to SUMO

Several other traffic simulation tools can serve as alternatives to SUMO, each with different features, advantages and limitations :

1. PTV-Vissim is a widely used microscopic traffic simulation software that also supports multi-modal transport systems. It offers a broad set of features comparable to those of SUMO. However, it is a proprietary tool that requires a commercial license, which can restrict its accessibility and limit the ability to implement custom modifications at the code level.
2. Aimsun is a commercial traffic simulation platform that supports microscopic, mesoscopic and macroscopic modeling. While it is used in various contexts, its commercial nature introduces licensing costs and reduces flexibility for developers who require deeper code-level access and customization.
3. CityFlow [33] is an open-source microscopic simulator designed for efficient, large-scale urban traffic simulations, capable of handling thousands of intersections with high computational efficiency. Its architecture is highly optimized for speed and parallel execution, making it over twenty times faster than SUMO in large-scale benchmarks [34]. CityFlow is particularly popular in the field of artificial intelligence and reinforcement learning, serving as a benchmark for multi-agent adaptive traffic signal control and dynamic routing tasks. However, CityFlow remains limited compared to SUMO in certain areas: it offers fewer out-of-the-box features for conventional traffic simulation, has a less mature ecosystem and does not natively support the simulation of public transportation modes such as buses, trams, or trains, which restricts its suitability for comprehensive, multi-modal urban simulations.

2.2.5 Choice of SUMO for this thesis

SUMO was selected for this project based on the following criteria:

- **Open-source** : SUMO's open-source status allows users to freely modify, adapt and extend the software, making it suitable for research and academic use where flexibility and transparency are important. This degree of code-level access is not available in proprietary tools like PTV Vissim or Aimsun.
- **Community and documentation** : SUMO benefits from a large active community

that contributes regularly to new features or bug fixes. In addition, it offers extensive documentation and a wide range of tutorials, which significantly lower the learning curve for new users.

- **Microscopic and multi-modal simulation** : SUMO provides detailed agent-based (microscopic) modeling of cars, public transport and pedestrians. While other simulators such as Vissim and Aimsun also support microscopic and multi-modal simulation, SUMO’s implementation is open, which enables deeper customization and integration for research needs.

- **Integration and flexibility** :

SUMO’s design and command-line tools (like `netconvert` and `duarouter`) facilitates working on various data sources making it easy to connect with external components.

Commercial tools like PTV Vissim and Aimsun offer advanced graphical user interfaces and support, however, their proprietary nature limits customizability and long-term usability in research settings. CityFlow, while open-source and scalable, lacks SUMO’s extensive documentation and wider range of traffic simulation features relevant for this project’s objectives.

2.3 Stochastic simulation

In a traffic-management digital twin, uncertainty is part of the phenomenon being modeled rather than a secondary implementation detail. Traffic demand, vehicle insertion, lane changing, speed preferences and route choices all contain stochastic components and small changes in these components can propagate into visibly different congestion patterns [9, 35, 32, 36]. In the United States, the Federal Highway Administration (FHWA) therefore recommends that microsimulation alternatives be evaluated with multiple random seeds because stochastic model variation can affect the statistical validity of observed differences between alternatives [36].

Stochastic simulation treats selected inputs or model mechanisms as random variables and observes the distribution of outcomes over repeated replications [37, 38]. For this work, the decision output is therefore not a single deterministic delay value for each plan, but a set of paired delay, travel-time and impacted-user observations across simulation seeds. The evaluator uses these observations to estimate means, confidence intervals and ranking robustness.

2.3.1 Theoretical foundations of the Monte Carlo method

The Monte Carlo method estimates quantities of interest by repeated random sampling. If each simulation run is treated as one realization of an output metric, then repeated runs approximate the distribution of that metric under the model’s stochastic mechanisms.

Law of Large Numbers and convergence. Let $X_1, \dots, X_n$ be independent replications of an output metric X with finite mean μ . The sample mean $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ converges to μ as n increases. In the traffic-simulation setting, X_i is the metric produced by run i with a distinct random seed. This is why additional replications stabilize the estimated mean, provided the model configuration is otherwise held fixed [37, 38].

Confidence intervals. A sample mean alone does not communicate how much uncertainty remains after n runs. For a metric with sample standard deviation s , the usual small-sample interval for the mean is

$$\bar{X}_n \pm t_{0.975, n-1} \frac{s}{\sqrt{n}},$$

where $t_{0.975, n-1}$ is the Student- t critical value. This is the form used in the evaluator’s 95% confidence intervals and is also the basis of the run-count formulas used in traffic microsimulation guidance [37, 39].

Paired design for plan comparison. A deviation plan and its baseline are compared under the same seed whenever possible. In simple terms, this means that the plan and the baseline are tested on the same simulated *traffic day*: the same random seed is used so that random effects such as vehicle insertion times or route-choice perturbations are aligned as much as possible. This matters because some seeds naturally create easier or harder traffic conditions. If a baseline run uses an easy seed and a deviation-plan run uses a difficult seed, the observed difference may come from the random draw rather than from the plan itself.

The paired design therefore compares each plan run only with the baseline run that uses the same seed. The estimator is applied to the run-by-run difference

$$\Delta^{(r)} = X_{\text{plan}}^{(r)} - X_{\text{baseline}}^{(r)}$$

rather than to two unrelated samples. Here, X can be a metric such as delay or mean travel time, and $\Delta^{(r)}$ asks a direct question for seed r : *how much did the plan change the metric relative to the comparable baseline?* Preserving this run-by-run difference removes part of the noise caused by random simulation conditions and leaves a clearer estimate of the plan effect. The paired series is then summarized with the same confidence-interval machinery used for other stochastic simulation outputs.

2.3.2 Randomness in microscopic traffic simulation

SUMO documents stochasticity as a core mechanism for reproducing reality in a simulation scenario. The seed initializes the random-number generators and a fixed seed makes a run deterministic and reproducible. Changing the seed changes the random sequence used by stochastic model components.

Several SUMO mechanisms are directly relevant here. **Speed distributions** allow vehicles to draw heterogeneous speed preferences through the `speedFactor` parameter, SUMO states that this random value is selected when each vehicle is created and that a default speed distribution is active. **Car-following stochasticity** enters through model parameters such as `sigma`, which introduces random variation in driver speed behavior. **Departure-time variation** can be added with random offsets or randomized flows, which prevents unrealistically synchronized departures and better represents variable demand realization [32].

2.3.3 Determining the number of simulation runs

Choosing the number of simulation runs is difficult because the required number is not known in advance. It depends on the variability of the network, the metric studied, the desired precision and the performance gap between candidate plans. Existing traffic-engineering guidance treats repeated random seeds as a way to capture stochastic variation and make uncertainty visible and sizes the budget so that a mean is estimated to a stated precision [38, 36, 39].

Comparing alternatives is a weaker requirement: the decision needs to know *which* plan is best, not the exact delay of each. Simulation-optimisation treats this as a *ranking and selection* problem, allocating a fixed budget of replications across K alternatives so as to maximise the *probability of correct selection* [40]. Equal allocation is generally not efficient: runs spent on an already clearly inferior alternative add little, whereas alternatives with high variance or a small gap to the leader are the ones still in doubt. Optimal Computing Budget Allocation formalises this and reports large budget reductions for a given selection probability [41] and greedy sequential procedures achieve comparable sample efficiency when the number of alternatives is large [42].

This matters here because each replication costs one `duarouter` call plus one SUMO run per candidate plan. The present work does not implement a sequential allocation rule: it uses a fixed budget, relies on the paired design for variance reduction and keeps all per-run observations so that the remaining uncertainty can be inspected and the budget extended when the ranking is still unstable (Section 3.4.5).

2.4 Multiple-criteria decision analysis

Multiple-criteria decision analysis (MCDA) supports decisions where alternatives must be evaluated on several criteria that are not naturally reducible to one physical unit. In applied MCDA, the decision process is made explicit by defining criteria, assigning scores, choosing weights and showing how these elements produce a ranking [43]. This reference is used here for that methodological structure, not as evidence about traffic behavior. The same structure fits deviation-plan evaluation: network delay, mean travel time, throughput feasibility and impacted-user burden are all relevant, but they do not express the same dimension of performance.

In this thesis, MCDA is not used to hide the trade-off inside one unexplained number. Instead, it is used to make the decision rule auditable. The evaluator first removes infeasible or dominated alternatives, then ranks the remaining plans with an additive score and finally tests whether the ranking is sensitive to the chosen weights.

2.4.1 Pareto dominance and non-dominated plan sets

Pareto dominance provides a weight-free screening rule. A feasible plan a *Pareto-dominates* a feasible plan b if a is no worse than b on every criterion and strictly better on at least one. Grierson gives the same formal condition for Pareto-optimal solutions in a multi-criteria minimization problem [44].

The non-dominated plans form the Pareto front. Restricting scoring to this set has

a practical benefit: no choice of non-negative weights can justify selecting a plan that is uniformly worse than another feasible plan. This is particularly useful in deviation planning because a plan can look attractive on one metric while being worse on all others once paired simulation results are considered.

2.4.2 Additive weighted scoring

Additive weighted scoring is a simple and widely used aggregation method in multi-criteria decision analysis. Each alternative is evaluated on several criteria, whose values are first transformed onto a common scale. The overall score of alternative k is then computed as

$$S_k = \sum_q w_q z_{kq}, \quad (1)$$

where z_{kq} is the scaled performance of alternative k on criterion q and w_q is the weight assigned to that criterion. The weights are non-negative and usually sum to one.

This approach is valued for its transparency: the final ranking follows directly from the selected criteria and their weights [43]. When all criteria are formulated as costs, lower scores indicate better alternatives. Its main limitation is that it allows compensation between criteria, meaning that poor performance on one criterion may be offset by good performance on another.

2.4.3 Weight sensitivity and robustness analysis

The main limitation of additive scoring is that the weights are not measured by the simulator. They express a value judgement about what the decision maker considers more important. For example, after all metrics have been normalized, assigning a larger weight to network delay than to impacted-user burden means that the evaluation is allowed to prefer a plan that saves more overall delay even if it affects more individual users. Another planner could reasonably make the opposite choice. The same simulation outputs can therefore lead to different rankings if the priorities are different. [45].

In this work, weight robustness is estimated by drawing many admissible weight vectors from the simplex and recomputing the ranking. The sampling distribution is $\text{Dirichlet}(1, \dots, 1)$, which is uniform over the weight simplex [46]. The robustness score of a plan is the fraction of sampled weight vectors for which it ranks first. A high robustness score means that the recommendation does not depend strongly on the reference weight vector, whereas a low robustness score signals that the decision should be escalated to explicit stakeholder weighting.

3 Methodology

3.1 System overview and pipeline

The methodology follows the same order as the implemented workflow. The first step generates a synthetic, network-aware demand from the SUMO road network and the Traffic Analysis Zone (TAZ) layer described in Section 3.2.2. This demand is written as `trips.xml`: every trip has a departure time, an origin edge and a destination edge, while the full path is left open. The closure is then translated into candidate deviation plans. In the Monte Carlo stage, the same trip demand is routed repeatedly with `duarouter`, using randomized edge weights, before each deviation plan is applied and simulated in SUMO. The last step compares the plans using paired statistics and a multi-criteria decision rule. The functional organization of this workflow is summarized in Table 2, while Figure 3 shows the same pipeline as a data flow from inputs to final evaluation files.

3.1.1 Concepts and contribution boundary

The methods chapter combines standard transport concepts, external SUMO tools and code developed for this thesis. Table 1 defines the main terms before the individual stages are described.

Term	Meaning in this thesis
SUMO	<i>Simulation of Urban MObility</i> , the external microscopic traffic simulator used to execute the traffic runs. SUMO is used as a tool, not developed in this thesis.
<code>duarouter</code> ⁷	SUMO’s command-line router. It converts unrouted trips into explicit route files, this thesis uses its randomized edge-weight option to generate route-choice variation.
TAZ	<i>Traffic Analysis Zone</i> , a geographic zone used to aggregate origins and destinations before individual trips are instantiated on SUMO road edges.
OD matrix	<i>Origin-Destination</i> matrix. Entry (i, j) gives the number or share of trips from origin zone i to destination zone j .
Gravity model [47]	A standard demand-allocation model from transport planning. The thesis contribution is the synthetic implementation: zone-mass proxy, sampled TAZ-to-TAZ impedance and integer hourly OD matrices.
Deviation plan	A candidate detour path around the closed edge or edge sequence. The plan objects and route-rewriting policy are implemented in this thesis.
Baseline	The no-detour, no-closure condition for the same demand and Monte Carlo seed. Each candidate plan is compared with its paired baseline run.

Term	Meaning in this thesis
Monte Carlo run	One repeated simulation under a fixed trip set but a different routing seed and randomized edge-cost realization.
BFS	<i>Breadth-first search</i> , a standard graph search used here to find upstream candidate detour origins by hop depth.
MCDA	<i>Multi-criteria decision analysis</i> , the final ranking layer that combines time-loss delay, mean travel time and impacted-user delay after feasibility and Pareto screening.

Table 1: Key concepts used in the methodology and the boundary between reused methods/tools and thesis-specific implementation choices.

The core contribution is therefore not a new traffic simulator, a new gravity model or a new shortest-path algorithm. It is the integration of these standard components into a reproducible pipeline that generates synthetic demand, computes candidate detours, applies those detours to route files and evaluates them across paired stochastic SUMO simulations.

Table 2 identifies the four implemented modules, their role in the workflow and the artifacts they write for the following stage. The table should be read as the functional view of the system: it explains what each module is responsible for before the following subsections describe the internal methods in more detail.

Module	Main role	Main outputs
<code>scenario_generator</code>	Generates synthetic OD demand and unrouted trip records	<code>trips.xml</code> , OD matrices
<code>deviation_plan_maker</code>	Converts the network into a graph and calculates the deviation plan(s)	JSON of candidate deviation plans
<code>monte_carlo_simulation</code>	Re-routes trip demand and simulates baseline and detour conditions over multiple runs	<code>run_#</code> folders, <code>summary.json</code>
<code>evaluator</code>	Paired statistical analysis and multi-criteria ranking	<code>evaluation.json</code> , <code>evaluation.md</code>

Table 2: Functional organization of the pipeline.

Figure 3 complements the table with the execution order. The dashed box contains the three inputs shared by the workflow, the vertical arrows show the order in which modules are executed and the side boxes show the main files produced along the way.

⁷<https://sumo.dlr.de/docs/CLI.html>

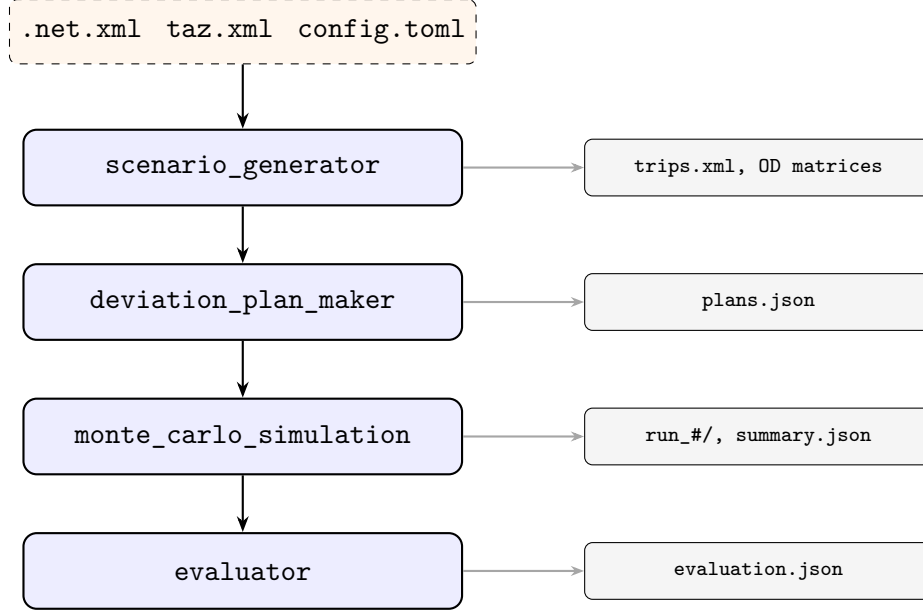


Figure 3: Four-module pipeline. Inputs (dashed box) flow top-to-bottom through the four modules. The scenario generator writes unrouted trips, explicit route files are produced later inside each Monte Carlo iteration by duarouter.

3.2 Scenario generation

The scenario generator creates the demand used by the rest of the pipeline. In this work, the demand must satisfy four constraints:

1. **Network-aware:** Origins and destinations are sampled from actual SUMO road edges.
2. **Zone-aware:** Demand is expressed at the TAZ level.
3. **Temporally structured:** The daily profile includes morning and evening peaks.
4. **Directional:** Peak periods can concentrate trips from residential zones toward employment zones and back.

This module is thesis code. It does not claim to estimate real Brussels demand from counts, surveys or mobile-phone traces. Instead, it constructs a controlled synthetic demand that is consistent with the available network files and is rich enough to test the detour-generation and evaluation pipeline. The standard modelling components used for this purpose are identified below, and the thesis-specific choices are the proxies, normalisations and sampling procedures used to turn them into reproducible SUMO inputs.

3.2.1 Module overview

The generation is split into three stages:

1. **Stage 1 - Demand generation:** construct 24 hourly Origin–Destination (OD) matrices using a gravity model [47] with an optional directional commute overlay.
2. **Stage 2 - Route library:** build a small set of candidate routes for each active OD pair using Yen’s k -shortest paths algorithm [48].

3. **Stage 3 - Trip instantiation:** assign a departure time and valid endpoint edges to every trip. The full route is assigned later, inside the Monte Carlo stage.

3.2.2 Stage 1 : Demand generation

Stage 1 produces hourly OD matrices, not individual routes. The output of this stage is a set of zone-to-zone trip counts for each hour of the day. Individual departure times and SUMO edge endpoints are assigned only in Stage 3.

Traffic Analysis Zones

The spatial unit of the model is the *Traffic Analysis Zone* (TAZ), a standard construct in transport demand modelling that aggregates individual trip origins and destinations into discrete geographic areas. TAZs in SUMO are defined as polygons that can be associated with the road network. Each TAZ is defined by the set of SUMO road edges it contains.

A scalar *zone mass* m_i is assigned to each TAZ as the capacity of its drivable edge lengths :

$$m_i = \sum_{e \in \mathcal{E}_i} \ell(e) \cdot n(e) \quad (2)$$

where $\ell(e)$ is the length of edge e in meters and $n(e)$ is its lane count and $\mathcal{E}_i$ is the set of edges in TAZ i .

Equation (2) should therefore be interpreted as an infrastructure-based proxy for the relative importance of each TAZ, rather than as a calibrated measure of real travel demand. The formula only accounts for the total drivable road space inside a zone, through edge lengths and lane counts. The underlying assumption is that zones containing more and larger road infrastructure are likely to support more traffic activity and can therefore be assigned a larger synthetic production or attraction mass. This approximation is used because no calibrated socio-economic productions or attractions, such as population, employment, land use, or observed trip data are available for this synthetic setting. In a classical four-step transport model, these quantities would instead be estimated from socio-economic data [17]. Here, lane kilometres⁸ are used as a practical surrogate: they are directly available from the network and major road corridors generally carry more traffic than minor links. If observed productions and attractions become available, they should replace this proxy.

Temporal Demand Profile

The total daily demand N is distributed across the 24 hours of the simulation using a *two-Gaussian profile*:

$$p_h = \frac{a_{AM} g(h; \mu_{AM}, \sigma) + a_{PM} g(h; \mu_{PM}, \sigma)}{\sum_{h'=0}^{23} [a_{AM} g(h'; \mu_{AM}, \sigma) + a_{PM} g(h'; \mu_{PM}, \sigma)]}, \quad (3)$$

where

$$g(h; \mu, \sigma) = \exp\left(-\frac{(h - \mu)^2}{2\sigma^2}\right), \quad (4)$$

⁸A lane kilometre is one kilometre of road with one lane.

μ_{AM} and μ_{PM} are the morning and evening peak hours respectively and σ controls the temporal spread. The amplitudes a_{AM} and a_{PM} allow the two peaks to differ in magnitude. The default configuration uses $a_{AM} < a_{PM}$, representing a slightly larger afternoon peak, which is common in urban networks [49].

The continuous profile p_h is converted into hourly trip counts n_h with the largest-remainder method. This keeps the total exactly equal to N while staying as close as possible to the continuous proportions.

Figure 4 illustrates the mathematical structure of the two-peak profile before integer rounding.

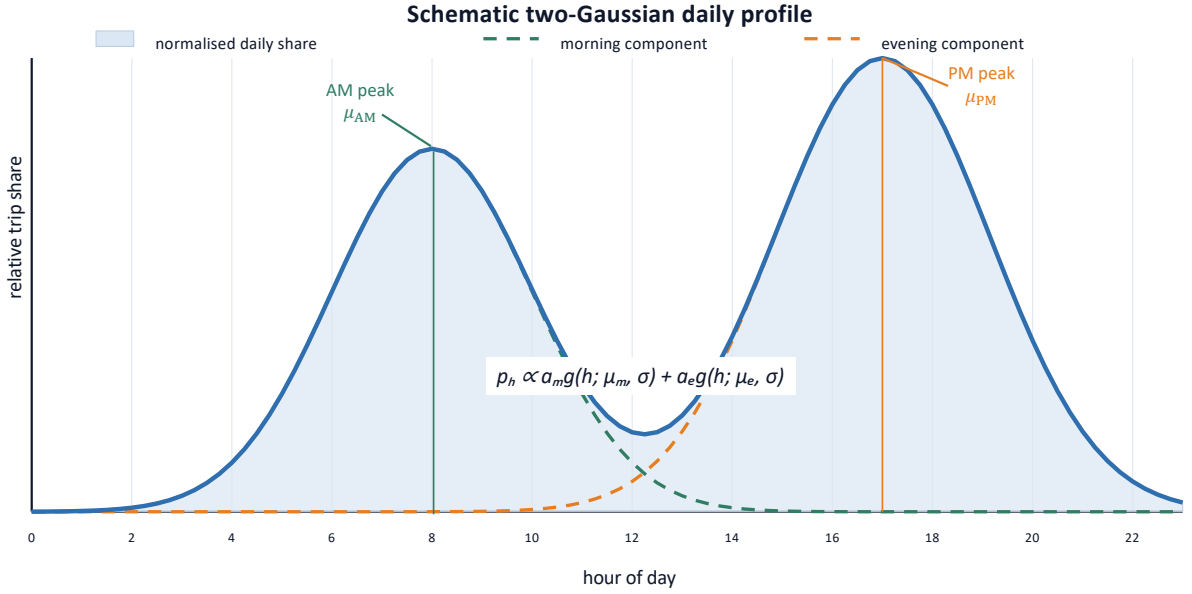


Figure 4: Illustrative two-Gaussian daily demand profile used to distribute the total daily demand across hours. The morning and evening components are centred on μ_{AM} and μ_{PM} , and the final hourly share is proportional to their weighted sum $a_{AM}g(h; \mu_{AM}, \sigma) + a_{PM}g(h; \mu_{PM}, \sigma)$.

Gravity OD model

The gravity model itself is not a new model introduced by this thesis. It is a standard transport-planning model used to distribute trips between origins and destinations as a function of origin mass, destination mass and travel impedance [47, 17]. What is specific to this work is its synthetic implementation: zone mass is approximated from lane kilometres, impedance is estimated from sampled shortest paths between TAZs and the resulting shares are converted into exact hourly integer OD matrices.

The spatial distribution of trips across TAZ pairs is governed by a *singly-constrained gravity model*. The **unnormalized** weight of the OD pair (i, j) is:

$$w_{ij} = m_i \cdot m_j \cdot \exp(-\beta c_{ij}), \quad i \neq j, \quad (5)$$

where c_{ij} is the travel-cost impedance between zones i and j (shortest path length in meters) and $\beta \geq 0$ is the distance decay parameter⁹.

⁹A larger β means trips decay faster with distance/cost (people are more reluctant to travel far), a

Pairs with infinite or negative impedance receive zero weight. The normalized *OD share matrix* is then:

$$S_{ij} = \underbrace{\frac{m_i}{\sum_k m_k}}_{\text{origin share } O_i} \cdot \underbrace{\frac{m_j \exp(-\beta c_{ij})}{\sum_\ell m_\ell \exp(-\beta c_{i\ell})}}_{\text{conditional destination probability } p(j|i)}, \quad (6)$$

so that, by construction, $\sum_j S_{ij} = m_i / \sum_k m_k$ and $\sum_{ij} S_{ij} = 1$.

For each hour h , the share matrix is scaled to the corresponding integer total n_h with the largest-remainder method, giving an exact integer OD matrix T^h with $\sum_{ij} T_{ij}^h = n_h$.

The model is *singly-constrained on the production side*. In trip-distribution terminology, productions are the trips generated by each origin zone, whereas attractions are the trips received by each destination zone. A production-side constraint therefore fixes the total fraction of trips that must leave each origin. In Equation (6), this production total is set by the origin mass share $m_i / \sum_k m_k$: after the destination probabilities for origin i are normalised, they must sum to one, so the whole row must sum to

$$\sum_j S_{ij} = \frac{m_i}{\sum_k m_k}$$

regardless of the destinations available from that origin. Equivalently, in an hour with n_h trips, origin i is assigned approximately $n_h m_i / \sum_k m_k$ outgoing trips before integer rounding.

The destination side is different. The term $m_j \exp(-\beta c_{ij})$ still affects how the trips from origin i are distributed across destinations: larger and closer destination zones receive higher conditional probability. However, no target total is imposed on the number of trips ending in destination j . The column sums $\sum_i S_{ij}$ are therefore left unconstrained and emerge from the aggregation of all origin rows. This is why the unnormalised weight $w_{ij} = m_i m_j \exp(-\beta c_{ij})$ can contain both origin and destination masses while the final model is still only constrained on the origin, or production, side.

In Wilson's balancing-factor formulation [17], this corresponds to enforcing only the origin-side factor A_i and fixing the destination-side factor $B_j \equiv 1$. A doubly-constrained model would additionally enforce observed zonal attractions and would require both balancing factors to be recovered by iterative proportional fitting. Since zonal attractions are not observed in the present synthetic setting, enforcing them would add parameters that cannot be identified from the available data. For this reason, the singly-constrained formulation is used here: it relies only on the information present in the network (zone masses and impedances) and lets destination totals emerge from the destination masses and distance-decay term $\exp(-\beta c_{ij})$. Figure 5 gives a schematic view of the same directed OD relations in matrix form.

smaller β means the distribution is more spatially spread out. Wilson discusses this in the context of calibrating the model to observed data.[47]

zone-to-zone demand idea

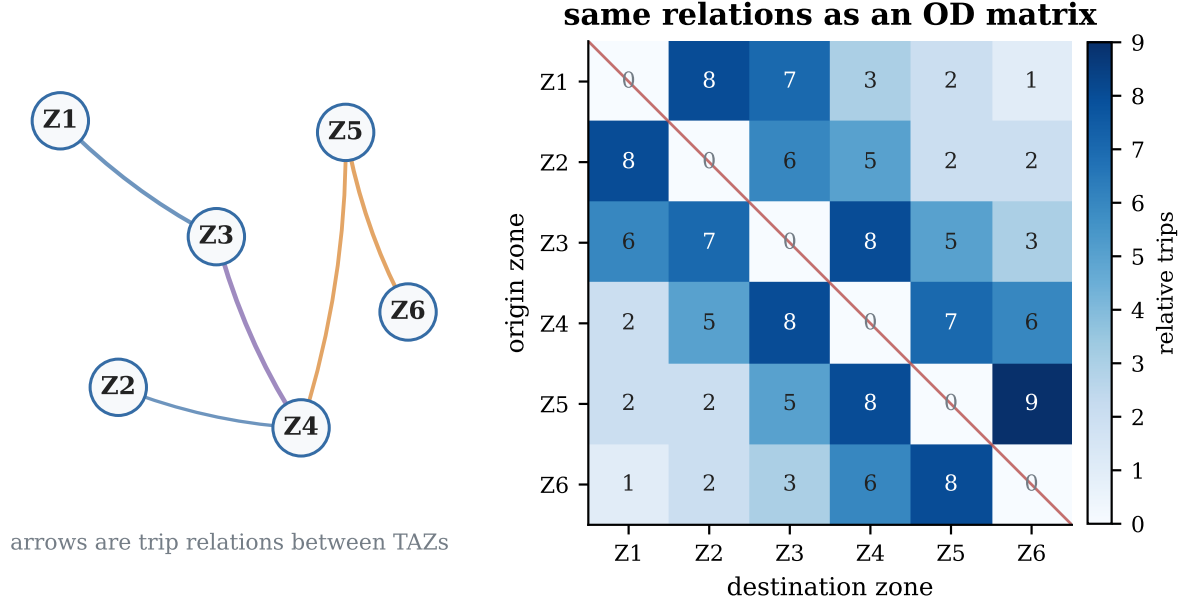


Figure 5: Schematic relationship between zone-to-zone trip relations and an OD matrix. Rows represent origin zones, columns represent destination zones, and each off-diagonal cell corresponds to the relative demand assigned to one directed OD pair. The diagonal is set to zero when intra-zonal trips are excluded.

Commutate Overlay (Optional)

When residential weights r_i and employment weights e_i are available, the hourly OD shares can be tilted toward commuting patterns. Two additional gravity matrices are built: one for the morning peak (AM), from residential zones to employment zones and one for the evening peak (PM), in the reverse direction:

$$w_{ij}^{\text{AM}} = m_i m_j f(c_{ij}) r_i e_j, \quad (7)$$

$$w_{ij}^{\text{PM}} = m_i m_j f(c_{ij}) e_i r_j, \quad (8)$$

where $f(c_{ij}) = \exp(-\beta_c c_{ij})$ discounts longer or costlier trips. In the AM matrix, r_i and e_j increase flows leaving residential zones and arriving at employment zones. In the PM matrix, the same logic is reversed.

The two directional matrices are blended with the base OD share matrix through a time-varying mixture. For each hour h , the blended share is:

$$S_{ij}^h \propto S_{ij}^{\text{base}} + \lambda g(h; \mu_m, \sigma) S_{ij}^{\text{AM}} + \lambda g(h; \mu_e, \sigma) S_{ij}^{\text{PM}}, \quad (9)$$

where $g(h; \mu, \sigma)$ is a Gaussian kernel centred on hour μ with spread σ . The parameter λ (**peak_weight**) controls how strongly the commute pattern modifies the background gravity model. Setting $\lambda = 0$ recovers the base model. After blending, the shares are normalised before being converted into integer trips. Figure 6 illustrates how the AM and PM terms tilt the base gravity share in opposite commute directions.

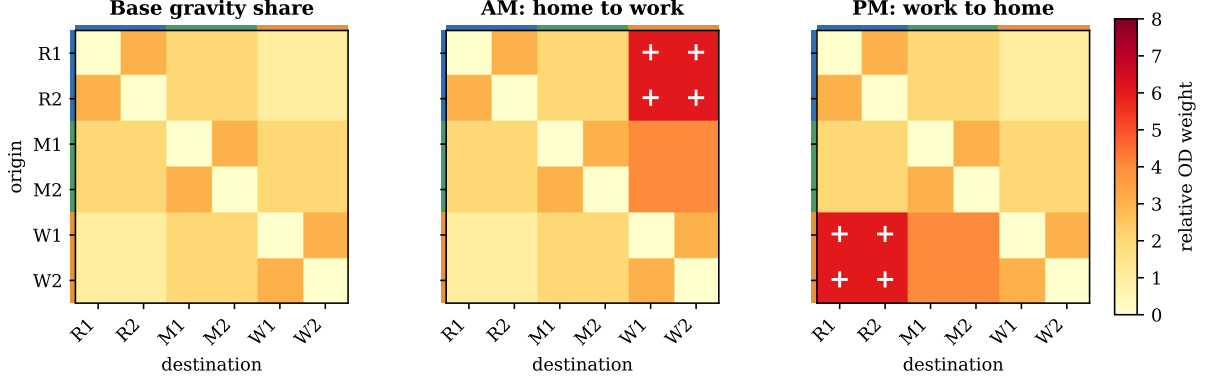


Figure 6: Illustrative commute overlay applied to the base gravity share. The AM term increases residential-to-workplace OD weights, while the PM term reverses the direction. In the implementation these directional matrices are blended with the base share through Equation 9 and then renormalised.

3.2.3 Stage 2 : Route library

Edge expanded network

Routing for the scenario generator uses an *edge-expanded* graph: SUMO road edges are the graph nodes and a directed arc connects edge u to edge v when SUMO allows the movement from u to v . The weight of this arc is the length of v , so a path cost corresponds to the sum of the edge lengths in the route.

This representation is convenient for two reasons. First, the generator has to sample SUMO edge endpoints and `duarouter` later expands those endpoints into complete routes. Staying at edge level keeps the generated demand close to SUMO’s own trip format. Second, turning feasibility is already encoded by SUMO connections: if an arc exists, the turn is allowed. Internal junction edges¹⁰ and edges that do not permit private-vehicle access are excluded when the graph is built.

Impedance Estimation

The gravity model needs a TAZ-to-TAZ impedance matrix. Computing exact all-pairs shortest paths would be expensive and would add precision that is not needed at this stage. Instead, for each directed zone pair (i, j) , the generator samples a small number of origin-edge and destination-edge combinations from the two zones. The lowest observed shortest-path cost is used as the representative impedance. This is an approximation, but it gives the gravity model the spatial signal it needs: nearby and well-connected zones receive lower costs than distant or poorly connected ones.

k-Shortest Candidate Routes

For each active OD pair (i.e. $T_{ij}^h > 0$ for at least one hour h), a set of at most k candidate routes is computed as follows:

1. Sample s concrete (origin-edge, destination-edge) pairs, where edges are drawn propor-

¹⁰SUMO ids beginning with :

tionally to zone mass (Equation (2)).

2. For each pair, enumerate up to k shortest simple paths using Yen’s algorithm [48]
3. Deduplicate across all samples by edge sequence, if the same sequence appears more than once, retain the lowest-cost instance.
4. Return at most k candidates sorted by cost.

3.2.4 Stage 3 : Trip Instantiation

Once the hourly OD matrices and the route library are available, every integer trip becomes a SUMO `trip` record. Two quantities are sampled: the departure time and a feasible candidate path. Only the first and last edges of that path are kept, because the final route is intentionally left to `duarouter` in the Monte Carlo stage.

Departure Time

Trips within each one-hour interval $[b, b + 3600)$ are assumed to depart uniformly over that interval. For an OD cell containing n trips, departure times are drawn independently as:

$$t_1, \dots, t_n \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(b, b + 3600). \quad (10)$$

With only hourly totals available, this is the least committed assumption: there is no information that would justify placing trips at a more precise time within the hour.

Endpoint Selection

For an OD pair with candidate route set $\mathcal{R}_{ij}$, each trip first selects one candidate path uniformly at random:

$$P(r \mid \mathcal{R}) = \frac{1}{|\mathcal{R}|}, \quad r \in \mathcal{R}. \quad (11)$$

Only the first and last edges of the selected candidate are exported:

$$\text{from} = r_1, \quad \text{to} = r_{|r|}. \quad (12)$$

This produces an unrouted `trips.xml` file. In this mode, the route library is used only to choose feasible endpoints, it is not a fixed route assignment. The full path choice is delegated to `duarouter` inside the Monte Carlo loop, where edge costs are randomized independently for each run.

The departure-time and endpoint draws use one seeded random-number generator. As a result, the generated `trips.xml` is reproducible. This matters because the Monte Carlo experiment is meant to vary routing and simulation seeds, not the underlying trip population.

3.3 Deviation plan computation

After demand generation, the next step is to define how the network can bypass the closure. The `deviation_plan_maker` takes the SUMO network and a user-selected closure

as input, then returns candidate detour paths around the disrupted segment. These plans are not evaluated at this stage, they are passed unchanged to the Monte Carlo simulation.

This module contains the main detour-construction contribution of the thesis: it converts a closure selected on the SUMO network into explicit detour-plan objects that can later be spliced into route files. The graph-search routines inside this module are standard algorithms. Dijkstra’s algorithm is used for shortest paths, Yen’s algorithm is used to enumerate loopless alternatives and breadth-first search (BFS) is used to discover upstream origins. The thesis contribution is how these algorithms are combined with closure-endpoint promotion, blocked-edge handling and depth-aware route rewriting.

Formally, let $G = (V, E, \ell)$ be a weighted directed graph in which V is the set of junctions, E the set of road edges and $\ell : E \rightarrow \mathbb{R}_{>0}$ an edge length function. Given a set of closed edges $E_c \subset E$ and a blocked set $B \supseteq E_c$, a *deviation plan* is defined as a tuple

$$\mathcal{D} = (s, t, P, \ell(P), d), \quad (13)$$

where $s \in V$ is the *origin* of this particular plan (the junction at which the detour begins) and $t \in V$ is the detour target, while $P = (e_1, \dots, e_m)$ is a loopless path in $G' = (V, E \setminus B)$ from s to t and $\ell(P) = \sum_{i=1}^m \ell(e_i)$ is its total geometric length in meters. The integer $d \in \mathbb{N}_0$ is the *breadth-first search (BFS) depth* of s relative to the *effective detour source* returned by the endpoint computation of Section 3.3.2. In the common case this effective source coincides with the upstream junction of the first closed edge $s_0 = \text{from}(e_1^c)$. If that junction has no usable alternative exit, the endpoint computation moves the source further upstream and adds the traversed prefix edges to the blocked set B . In the upstream method, the origin s can differ from one plan to another: the effective source, t and B are shared, but individual plans may start further upstream. When $d = 0$, the plan starts at the effective source and acts as the shallowest fallback. Plans with $d > 0$ start further upstream and are tried first during multi-plan route rewriting.

3.3.1 Junction-level graph representation

The `deviation_plan_maker` operates on a *junction-level* directed graph, in contrast to the edge-expanded representation used by the scenario generator in Section 3.2.3. Here nodes correspond to SUMO junctions and arcs correspond to road edges: an arc (u, v) exists for every edge whose `from` attribute is junction u and whose `to` attribute is junction v . Arc weights default to the geometric length of the edge in meters. An alternative weight can be loaded from a SUMO `edgedata.xml` file, in which case values are averaged across all measurement intervals and the resulting mean is stored as the weight, if the field encodes speed rather than travel time, the reciprocal is stored instead so that shorter weighted distances correspond to faster edges.

The graph is built directly from the `.net.xml` file. The implementation keeps three dictionaries (`edges`, `outgoing` and `incoming`) that provide $O(1)$ lookup of any edge record and $O(\deg(v))$ enumeration of a node’s neighbours¹¹. These structures are sufficient for the routing operations used in this module.

¹¹Here, $\deg(v)$ denotes the number of outgoing or incoming edges adjacent to node v , depending on the direction of the enumeration. Thus, neighbour enumeration is linear in the local branching factor of the queried junction, not in the total number of edges in the network.

3.3.2 Detour endpoint computation

Given a sequence of closed edges $E_c = (e_1, \dots, e_n)$, the natural candidate detour source is the upstream junction of e_1 and the candidate target is the downstream junction of e_n . However, if the source junction has only one outgoing edge (the first closed edge itself), no detour can diverge from it, the same problem arises symmetrically at the target if it has only one incoming edge.

To handle this, Algorithm 1 walks upstream from the source and downstream from the target until it reaches junctions with at least two outgoing (resp. incoming) edges. Every intermediate edge encountered during these walks is added to the blocked set, so the router cannot reuse it as a shortcut.

Algorithm 1 FINDDETOURENDPOINTS

Require: closed edge sequence E_c , graph G

Ensure: source s , target t , extended blocked set B

- 1: start with s and t at the upstream and downstream ends of the closure
 - 2: initialise B with the closed edges
 - 3: **while** s has only one way out (the closure itself) **do**
 - 4: step one junction further upstream and add the traversed edge to B
 - 5: **end while**
 - 6: **while** t has only one way in **do**
 - 7: step one junction further downstream and add the traversed edge to B
 - 8: **end while**
 - 9: **return** s, t, B
-

Figure 7 illustrates the procedure on a small graph in which the natural endpoints s_0 and t_0 are both dead ends and the walks promote s and t as the actual detour endpoints.

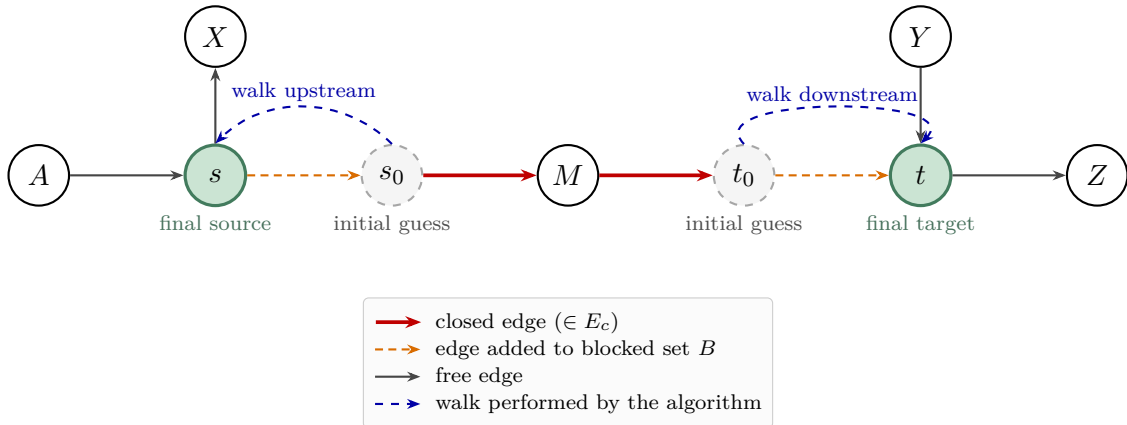


Figure 7: Detour endpoint computation on an illustrative graph. The closure $E_c = (s_0 \rightarrow M, M \rightarrow t_0)$ is shown in red. Because s_0 has only one outgoing edge (the closure itself) and t_0 has only one incoming edge, neither can serve as a detour endpoint. The algorithm walks one junction upstream from s_0 to s (which has the additional outgoing edge $s \rightarrow X$) and one junction downstream from t_0 to t (which has the additional incoming edge $Y \rightarrow t$). The two traversed edges (orange, dashed) are appended to the blocked set B so they cannot be reused by the router as shortcuts around the closure.

3.3.3 Candidate generation

Two candidate-generation methods are implemented. Both start from the same endpoint computation, but they use it differently. Yen’s method keeps one detour source s and enumerates the k best loopless paths from s to t , all of its plans have depth $d = 0$. The upstream method starts from the same effective source, then searches backwards up to $d_{\max}$ hops. Each upstream node becomes a possible plan origin and the shortest path from that origin to t is retained if it exists. Yen’s method is the default in the end-to-end pipeline. The upstream method is used when the experiment explicitly needs detours that begin before the closure point.

Yen’s k -shortest loopless paths.

With source s , target t and blocked set B fixed, the first method calls Yen’s k -shortest loopless paths algorithm [48] to enumerate up to k alternative routes from s to t that avoid every edge in B .

As described in Algorithm 2, Yen’s algorithm extends Dijkstra’s shortest-path search by iteratively generating *spur paths*. Let P_1 be the shortest path returned by Dijkstra. At each subsequent iteration i , the algorithm considers every prefix of the previously accepted path P_{i-1} as a potential root segment. For each root, it temporarily blocks the outgoing arc that was used by any already-accepted path sharing that root, then runs Dijkstra from the spur node (the last node of the root) to t . The resulting spur path is concatenated with the root to form a complete candidate. All candidates are stored in a min-heap keyed by total cost and the minimum is promoted to P_i . A global signature set prevents the same edge sequence from appearing twice. Heap entries carry a monotonically increasing tie-breaking counter, so that two paths with identical cost are ordered by insertion time rather than by a potentially fragile lexicographic comparison of node lists.

Algorithm 2 YENKSHORTESTPATHS

Require: graph G , source s , target t , integer $k \geq 1$, blocked set B

Ensure: list $\mathcal{P}$ of up to k diverse shortest paths from s to t avoiding B

```

1:  $P_1 \leftarrow$  shortest path from  $s$  to  $t$  avoiding  $B$ 
2:  $\mathcal{P} \leftarrow [P_1]$ 
3: for  $i \leftarrow 2$  to  $k$  do
4:   candidates  $\leftarrow \emptyset$ 
5:   for each prefix of the previously accepted path  $P_{i-1}$  do
6:     temporarily block the next arc taken by any already-accepted path that shares
       this prefix
7:     find the shortest path from the prefix’s endpoint to  $t$ 
8:     store prefix + new path as a candidate
9:   end for
10:  pick the cheapest unseen candidate as  $P_i$  and append it to  $\mathcal{P}$ 
11:  stop if no candidate remains
12: end for
13: return  $\mathcal{P}$ 

```

Upstream search.

The upstream method is based on the fact that vehicles approaching a closure may arrive from several directions, not from one junction only. A reverse BFS from s records every reachable upstream node u and its depth $d(u)$ up to $d_{\max}$. The source s itself is kept at depth 0, so the method still contains a fallback. For each discovered origin, Dijkstra is run from that origin to t while avoiding the blocked set B .

The returned plans are sorted by depth descending and then by total length. This ordering is used later by the route rewriter: it tries the most upstream applicable plan first, then falls back to shallower ones. Algorithm 3 summarizes this reverse search, path computation and sorting step.

Algorithm 3 UPSTREAMDEVIATIONPLANS

Require: graph G , endpoints s, t , blocked set B , max depth $d_{\max}$

Ensure: deviation plans sorted by depth (desc), then length (asc)

```
1: reverse-BFS from  $s$  up to  $d_{\max}$  hops, tagging each reachable node  $u$  with its depth  $d(u)$ 
2:  $\mathcal{P} \leftarrow []$ 
3: for each upstream origin  $u$  found (including  $s$  at depth 0) do
4:   compute the shortest path  $\pi$  from  $u$  to  $t$  avoiding  $B$ 
5:   if  $\pi$  exists and is not a duplicate then
6:     save  $\mathcal{D}(\pi, d(u))$  as a candidate plan in  $\mathcal{P}$ 
7:   end if
8: end for
9: sort  $\mathcal{P}$ : deeper origins first, shorter paths first within each depth
10: return  $\mathcal{P}$ 
```

where $\mathcal{D}(\pi, d)$ denotes the deviation plan wrapping path π with BFS depth d .

Figure 8 illustrates the two methods on the same toy network: nodes s and t are the detour endpoints, p and q are intermediate junctions and the direct edge $s \rightarrow t$ belongs to the blocked set B .

Each accepted path is wrapped in a **DeviationPlan** object. The object records the plan identifier, its rank within the k -shortest list, the ordered sequences of junction and edge identifiers and the total geometric length in meters (computed by summing the **length_m** attribute of every edge on the path).

Plans are not removed because of a maximum detour-length ratio and near-duplicates are not removed by edge-set overlap. This keeps the planning stage simple and avoids discarding a route before it has been tested in simulation. Two routes can share most of their edges and still behave differently if they diverge near a bottleneck or pass through a different sequence of signalised intersections. The Monte Carlo evaluation is used later to separate useful candidates from poor ones.

3.3.4 Route rewriting

Detours are applied by rewriting route files before SUMO starts, rather than by using SUMO's dynamic *rerouters* [50]. This keeps the experiment static and inspectable: **duarouter** already provides route-choice variation through randomized edge weights and

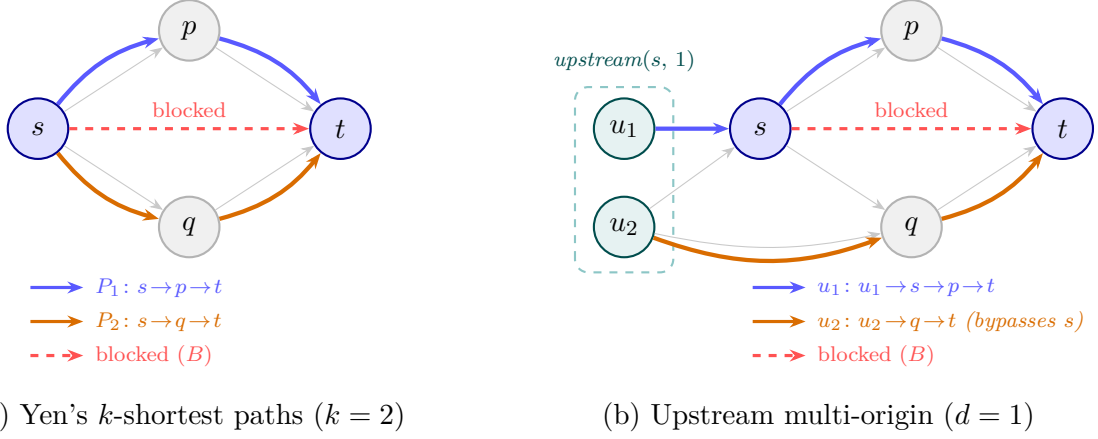


Figure 8: Comparison of the two candidate generation methods on a toy network. In both cases the edge $s \rightarrow t$ is blocked (red dashed). (a) Yen's algorithm produces $k = 2$ loopless paths, both originating from s : P_1 takes the upper corridor through p and P_2 the lower corridor through q . (b) The upstream BFS at depth $d = 1$ identifies two origins $\{u_1, u_2\}$ (teal). u_1 routes through s and then takes the upper corridor, while u_2 holds a direct connection to q , so its optimal path bypasses s entirely, a candidate that Yen's method, being anchored at s , would never generate.

the final route file can be checked before the simulation.

After each Monte Carlo call to `duarouter`, the rewriter creates one `routes_{plan_id}.xml` file per candidate plan. Only vehicles that traverse a closed edge are modified. For each affected vehicle, candidate plans are tried from deepest to shallowest and the first plan whose origin appears upstream of the closure in that vehicle's route is selected (Algorithm 4). Yen plans have only the evaluated depth-0 candidate, upstream plans also carry the depth-0 fallback.

The selected plan $\mathcal{D}^*$ is inserted by replacing the route segment from $\mathcal{D}^*.s$ to the last closed edge with the detour edge sequence $\mathcal{D}^*.P$. This is a splice in the vehicle's edge list.

Algorithm 4 DEPTHAWAREREWRITE

Require: vehicle route R , candidate plans $\mathcal{D}$ sorted by depth (desc), closed edge set B_c

Ensure: rewritten route R'

- 1: **if** R does not traverse any edge in B_c **then**
 - 2: **return** R *// route unaffected by the closure*
 - 3: **end if**
 - 4: pick the deepest plan $\mathcal{D}^*$ whose origin s lies on R upstream of the closure *// shallower plans provide deterministic fallback*
 - 5: splice $\mathcal{D}^*.P$ into R , replacing the segment that runs from the edge departing $\mathcal{D}^*.s$ up to the last closed edge
 - 6: **return** the spliced route
-

The baseline route file is left unchanged, each candidate is evaluated through its own rewritten copy.

Figure 9 illustrates this route-list splice for one affected vehicle.

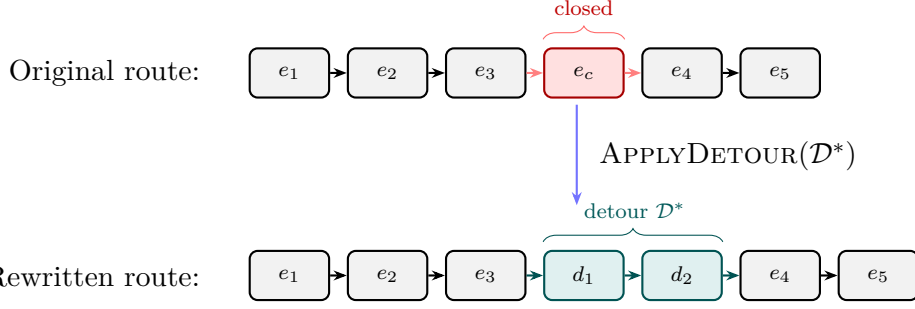


Figure 9: Route rewriting for one affected vehicle. The closed edge e_c (red) is replaced by the selected detour sequence (d_1, d_2) (teal), while the unaffected prefix and suffix are preserved.

3.4 Monte Carlo Simulation

The Monte Carlo stage turns the fixed demand and the candidate detours into repeated SUMO observations. In this stage, the demand stays fixed while the route assignment changes. The scenario generator gives the simulation an unrouted `trips.xml` file and each Monte Carlo iteration routes that same trip set again with `duarouter`, a different seed and randomized edge weights.

Here, a Monte Carlo run means one repetition of the same experiment under a different routing random seed. The baseline in each run is the route realization without applying any detour plan or closure treatment. Candidate plans are then applied to a copy of that same baseline route file. This paired design means that a plan is compared against the exact route-choice realization from which it was produced, rather than against an average from other seeds.

3.4.1 Experimental design

The experiment uses a **paired stochastic-routing design**. The trip population is fixed, while the route realization changes from run to run. Let $\mathcal{T}$ be the `trips.xml` file produced by the scenario generator. For Monte Carlo iteration r , the simulation first computes the baseline route file

$$\mathcal{R}_0^{(r)} = \text{DUAROUTER}(\mathcal{T}, G, r, \alpha), \quad (14)$$

where G is the SUMO network and α is the upper bound passed to `--weights.random-factor`. With this option, `duarouter` perturbs edge costs by drawing multipliers in $[1, \alpha]$. This does not change the trip origin or destination, it changes only how attractive each road segment appears during route computation. In human terms, this represents a driver who still wants to reach the same destination but may slightly adapt the chosen path after observing or anticipating traffic on particular streets. Different iterations can therefore produce different paths for the same trip endpoints. Figure 10 illustrates this mechanism on a small two-route graph.

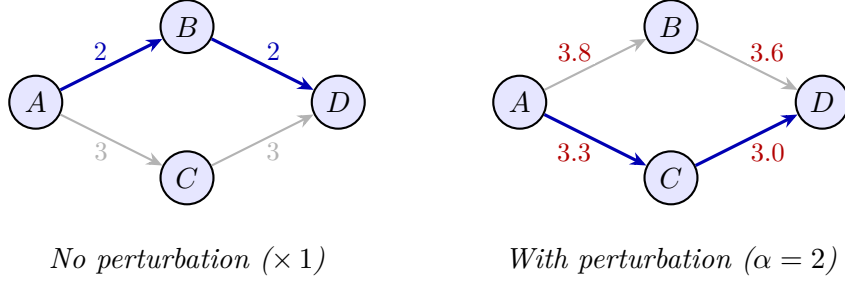


Figure 10: Effect of `-weights.random-factor` on route choice. Both graphs share the same network and origin–destination pair $A \rightarrow D$. **Left:** nominal edge costs, the router selects $A \rightarrow B \rightarrow D$ (cost 4) over $A \rightarrow C \rightarrow D$ (cost 6). **Right:** each cost is multiplied by a random factor drawn from $[1, \alpha]$, the top path becomes costlier (7.4) than the bottom path (6.3), so the router now chooses $A \rightarrow C \rightarrow D$. Across Monte Carlo iterations, different draws of these multipliers yield different route realizations for the same trip set.

For every candidate deviation plan $\mathcal{D}_k$, the deterministic rewriter is then applied to that same baseline route file. The baseline and all plan conditions are paired within the same run: they share the same trip set, the same route realization before applying the closure treatment and the same SUMO seed.

Meanwhile all available metrics are collected and saved for later evaluation. For each iteration r , the raw time-loss difference

$$\delta_k^{(r)} = D_k^{(r)} - D_0^{(r)} \quad (15)$$

measures the effect of plan $\mathcal{D}_k$ relative to the no-closure baseline under the same route-choice and SUMO random seed. The estimator

$$\bar{\delta}_k = \frac{1}{R} \sum_{r=1}^R \delta_k^{(r)} \quad (16)$$

compares plans after much of the run-level noise shared by the baseline and plan conditions has been removed.

3.4.2 Per-run routing and detour application

There is no plan-specific route pre-computation outside the Monte Carlo loop. The only fixed demand input is `trips.xml`. In each iteration, `duarouter` writes `run_#/routes.xml`. This is the baseline route realization for that seed. The route rewriter then creates the `routes_{plan_id}.xml` files in the same run directory.

After each plan file is written, the code checks that the closed edge no longer appears in any rewritten route. It also compares the baseline and plan files vehicle by vehicle to count how many routes changed. These statistics are stored per run, since the affected vehicles can change when `duarouter` produces a different baseline route set.

3.4.3 Iteration structure

Each Monte Carlo iteration consists of routing, deterministic route rewriting, SUMO execution and metric extraction. Algorithm 5 states the procedure formally.

Algorithm 5 MONTECARLOITERATION

Require: network file G , trip file $\mathcal{T}$, closed edge e_c , candidate plans $\{\mathcal{D}_1, \dots, \mathcal{D}_K\}$, edge-endpoint map $\mathcal{E}$, run index r , simulation window $[b, e]$, random factor α

Ensure: per-condition metric dictionary for this iteration

- 1: $\mathcal{R}_0^{(r)} \leftarrow \text{DUAROUTER}(\mathcal{T}, G, r, \alpha)$ $\triangleright$ baseline route realization
 - 2: $M_0 \leftarrow \text{RUNSUMO}(G, \mathcal{R}_0^{(r)}, r, b, e)$
 - 3: $\text{results}[\text{baseline}] \leftarrow M_0$
 - 4: **for** $k \leftarrow 1$ **to** K **do**
 - 5: $\mathcal{R}_k^{(r)} \leftarrow \text{APPLYDETOURTOROUTES}(\mathcal{R}_0^{(r)}, \mathcal{D}_k, e_c, \mathcal{E})$
 - 6: validate that $\mathcal{R}_k^{(r)}$ contains no closed edge e_c
 - 7: $s_k^{(r)} \leftarrow \text{COUNTREROUTED}(\mathcal{R}_0^{(r)}, \mathcal{R}_k^{(r)})$
 - 8: $M_k \leftarrow \text{RUNSUMO}(G, \mathcal{R}_k^{(r)}, r, b, e)$
 - 9: $\text{results}[\mathcal{D}_k] \leftarrow M_k \cup s_k^{(r)}$
 - 10: **end for**
 - 11: **return** results
-

DUAROUTER invokes `duarouter` with the fixed trip file, `-weights.random-factor` and the run seed. RUNSUMO writes a minimal SUMO configuration file that specifies the network, the route file, the simulation window $[b, e]$ and the same seed r , invokes `sumo` as a subprocess and parses the resulting `tripinfo.xml` output into the metrics of Section 3.4.4. Each condition runs in its own subdirectory (`run_r/baseline/`, `run_r/plan_k/`), keeping outputs isolated.

Figure 11 summarizes the same iteration as a data flow from fixed inputs to per-run baseline and plan outputs.

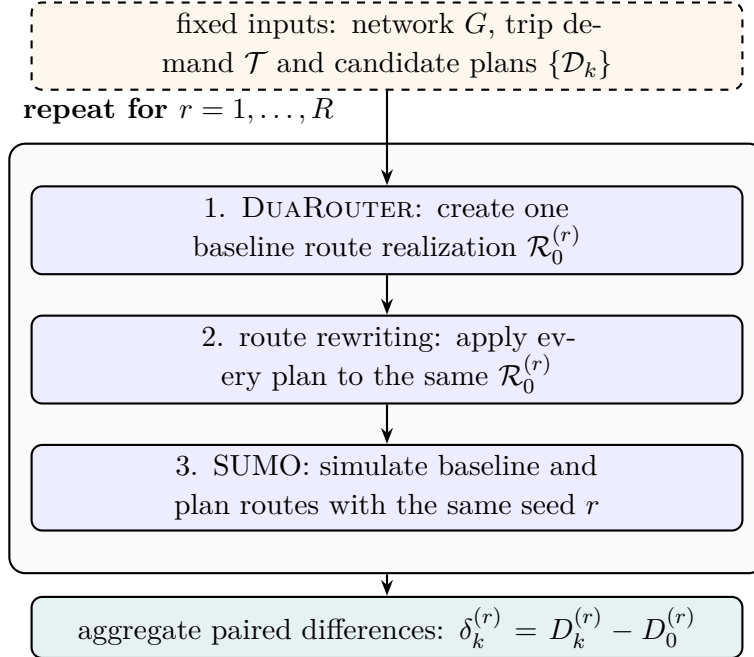


Figure 11: Monte Carlo protocol. The demand and candidate plans are fixed, each seed produces one baseline route realization, applies all detours to that realization and simulates paired baseline-plan conditions.

3.4.4 Performance metrics

Each `tripinfo.xml` file is parsed into a metric vector $M_c^{(r)}$ for condition c and run r . SUMO is configured with `tripinfo-output.write-unfinished`, so completed trips can be separated from unfinished or vaporized records. Aggregate performance is computed only on completed trips. The unfinished count is kept as a diagnostic.

Time-loss delay is the primary network-efficiency metric, but it is not the only assessment criterion. The Monte Carlo stage stores time-loss delay, mean travel time, completed trips, unfinished trips and the number of vehicles whose route changed after applying a plan. The evaluator then derives additional impacted-user metrics from the route and `tripinfo.xml` files.

- **Total time-loss delay** D (vehicle-hours): the sum of SUMO `timeLoss` over completed trips, converted from seconds to hours,

$$D = \frac{1}{3600} \sum_{v \in \mathcal{V}_{\text{done}}} \lambda_v, \quad (17)$$

where λ_v is the vehicle's SUMO `timeLoss`. This is the excess travel time relative to SUMO's ideal traversal of the driven route, so it captures both stopped waiting and moving slowdowns.

- **Mean travel time** $\bar{\tau}$ (minutes): the arithmetic mean of completed trip durations,

$$\bar{\tau} = \frac{1}{60 |\mathcal{V}_{\text{done}}|} \sum_{v \in \mathcal{V}_{\text{done}}} \tau_v, \quad (18)$$

where τ_v is the door-to-door duration of trip v in seconds.

- **Completed and unfinished trips**: the number of vehicles that arrived within the simulation window and the number of records that did not. A high unfinished count flags a simulation window, demand, or topology problem that should be inspected before interpreting a plan.
- **Rerouted vehicles**: for each plan and run, the number and percentage of vehicles whose route sequence differs from the run's baseline route file. This value is computed after `duarouter`, so it can vary across Monte Carlo iterations.

For plan comparison, the evaluator computes paired differences such as $\delta_k^{(r)}$ in Equation (15) run by run. These raw paired observations are preserved in the per-run `results.json` files and are not replaced by only top-level averages.

3.4.5 Convergence and number of runs

The number of runs R is fixed by the user. Since each run is computationally expensive (one `duarouter` call and $K + 1$ SUMO executions), choosing R is a real budget question. In the simulation literature this is studied as a *simulation budget allocation* or *ranking and selection* problem: rather than estimating every alternative's mean to equal precision, procedures such as Optimal Computing Budget Allocation distribute a limited number of replications so as to maximise the probability of correctly selecting the best alternative

[41, 40, 42]. Implementing such a sequential procedure is left as future work; the present design instead relies on two simpler safeguards. First, the paired design of Section 3.4.1 compares each plan with the baseline inside the same run, which cancels run-level noise shared by both conditions (a common random numbers effect) and therefore reduces the number of runs needed to separate the plans. Second, the summary output reports for each plan the mean paired delta together with its standard deviation and an approximate 95% confidence half-width across runs, so it can be verified a posteriori that the observed plan ranking is larger than the remaining statistical noise. When it is not, the resume mechanism extends the same output directory with additional runs, continuing from the next missing run identifier.

3.5 Evaluation and decision support

The last step is the evaluation of the simulated plans. At this point no new traffic is generated and no additional SUMO simulations are run. The task is to take the repeated outputs from the Monte Carlo experiment and compare the candidate detours in a way that is fair, reproducible and useful for the final discussion. The key point is to keep the paired structure of the experiment: for each Monte Carlo seed, the evaluator has one baseline result and one result for each candidate deviation plan. Each plan is compared against the baseline from the same run, not against an unrelated average.

The assessment is deliberately multi-metric. Total time-loss delay is treated as the primary network-efficiency metric, but it is not the only criterion used to judge a plan. Mean travel time, completed-trip feasibility and impacted-user delay are all kept in the evaluation, and the final ranking is produced by a multi-criteria decision analysis (MCDA) layer rather than by a single time-loss number.

In practice, the evaluation is organised around three questions:

- How much each plan changes the network-level performance compared with the baseline.
- Whether the plan remains feasible, in particular whether it avoids reducing the number of completed trips.
- Among the feasible plans, which alternatives still look reasonable after Pareto screening and a sensitivity check on the decision weights.

3.5.1 Choice of evaluator metrics

The evaluator does not try to prove that one detour is universally best. It separates the comparison into quantities that answer different planning questions:

- **Total time-loss delay** measures the network-level congestion burden created or removed by a plan. It is the primary efficiency metric because it captures both queues and slow movement relative to the vehicle’s ideal route traversal.
- **Mean travel time** measures the average trip burden for completed vehicles. It complements time-loss delay because a plan can reduce congestion on the chosen route while still making paths longer, or conversely add local congestion while shortening many trips.

- **Completed and unfinished trips** are used as a feasibility diagnostic. A plan must not appear good merely because more vehicles fail to finish within the simulation window and are therefore absent from completed-trip averages.
- **Fraction of impacted users delayed** focuses on the vehicles whose route actually changed. Network averages can hide a plan that performs well overall but imposes most of the cost on the directly affected users.

These metrics are also practical: they are available for every Monte Carlo run from SUMO's `tripinfo.xml` output and from the route comparison step, so the same definitions can be applied consistently to every candidate plan. The statistical tests reported by the evaluator are diagnostic checks on the paired effects, they are not the ranking rule. The final decision layer uses effect sizes, feasibility, Pareto dominance and weight sensitivity.

3.5.2 Comparable run set

The first operation is to identify the runs that can actually be compared. The evaluator scans the Monte Carlo output directory for `run_#/results.json` files and keeps a run only when it contains the baseline and all candidate plans listed in `summary.json`. Runs with missing plan outputs are left out of the paired analysis, but they are still recorded in the metadata so that the exclusion is visible.

This step avoids comparing one plan over more replications than another. After the filtering, every paired statistic is computed on the same run identifiers for every plan. If no complete run is available, the evaluator stops instead of producing a ranking based on incomplete evidence.

3.5.3 Paired network-level statistics

For every plan $\mathcal{D}_k$ and every retained Monte Carlo run r , the basic quantity is the difference between the plan and the baseline from the same run. For the primary network-level metric, this gives

$$\Delta D_k^{(r)} = D_k^{(r)} - D_0^{(r)}, \quad (19)$$

where $D_0^{(r)}$ is the baseline time-loss delay from run r and $D_k^{(r)}$ is the corresponding value after applying plan $\mathcal{D}_k$. For mean travel time, the same construction is used:

$$\Delta \bar{\tau}_k^{(r)} = \bar{\tau}_k^{(r)} - \bar{\tau}_0^{(r)}. \quad (20)$$

The sign convention is kept simple: a positive paired difference means that the plan worsened the metric in that run and a negative value means that it improved it. The full vector of paired differences is kept, not only the average, because the consistency of an effect matters as much as its mean.

For a generic paired-difference vector $x_1, \dots, x_n$, the evaluator reports

$$\bar{x}, \quad s, \quad \min(x), \quad \max(x), \quad \tilde{x}, \quad (21)$$

where $\bar{x}$ is the sample mean, s the sample standard deviation and $\tilde{x}$ the median. The mean estimates the expected effect of the plan over the seed distribution, the standard

deviation describes run-to-run variability and the median is kept as a robust centre when a few runs are extreme.

The confidence interval answers a different question: how precisely the Monte Carlo experiment has estimated the mean paired effect. Because the true standard deviation of the paired effects is unknown and must be estimated from the n replications, the normal critical value is replaced by the Student- t critical value. The Student- t distribution has heavier tails than the normal distribution, so it gives wider intervals when only a small number of runs is available. The evaluator uses the standard 95% level, corresponding to a two-sided 5% error rate, as a conventional compromise between intervals that are too narrow to be reliable and intervals that are too wide to be useful. The interval is computed as

$$\bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}, \quad (22)$$

when at least two runs are available. In repeated experiments satisfying the usual independence assumptions, this procedure would contain the true mean effect in about 95% of such intervals, it does not mean that there is a 95% probability that this particular realized interval contains the truth.

Two diagnostic hypothesis tests are reported for each paired metric. Both ask whether the observed paired effects are compatible with a zero-effect baseline. The paired one-sample t -test checks the null hypothesis

$$H_0 : \mathbb{E}[\Delta M_k] = 0, \quad (23)$$

where ΔM_k is the paired effect of plan k on the metric being tested. It is most appropriate when the paired differences are approximately normally distributed. A Wilcoxon signed-rank test is also reported as a non-parametric companion. It ranks the absolute non-zero paired differences, attaches the original signs and checks whether positive and negative ranks are balanced around zero. This makes the check less dependent on the normality assumption, which is useful for traffic simulation outputs with small samples or occasional extreme runs.

3.5.4 Feasibility gate

Before plans are ranked, they pass through a completed-trip feasibility check. A candidate is considered feasible only if it completes at least as many trips as the baseline in every retained run:

$$C_k^{(r)} \geq C_0^{(r)} \quad \forall r \in \{1, \dots, n\}, \quad (24)$$

where $C_k^{(r)}$ is the number of completed trips under plan $\mathcal{D}_k$. The reason for this rule is straightforward: a detour should not look better simply because more vehicles failed to arrive before the end of the simulation window. Plans that fail the gate are still shown in the report, but they are not allowed into the Pareto screening or weighted score.

3.5.5 Impacted-vehicle metrics

Network-level metrics are necessary, but they can hide what happens to the drivers who are directly affected by the closure. For this reason, the evaluation also considers an impacted

vehicle set. When route files are available, a vehicle is counted as impacted for a plan if its edge sequence differs between the baseline route file and the plan-specific route file:

$$\mathcal{I}_k = \{v : R_{k,v} \neq R_{0,v}\}. \quad (25)$$

If route-difference information is not available, the evaluator can fall back to the vehicles whose baseline route crosses the closed edge. This is less precise, but it still keeps the analysis focused on users directly connected to the disruption.

For each retained run, the impacted-user analysis compares only vehicles that are present and completed in both the baseline and plan `tripinfo.xml` files. It reports the total additional travel time for impacted users,

$$\Delta T_{k,\mathcal{I}}^{(r)} = \sum_{v \in \mathcal{I}_k} (\tau_{k,v}^{(r)} - \tau_{0,v}^{(r)}), \quad (26)$$

the mean effective speed over their trajectories and the *fraction of impacted users delayed*, written ϕ_{net} :

$$\phi_{\text{net},k}^{(r)} = \frac{|\{v \in \mathcal{I}_k : \tau_{k,v}^{(r)} > \tau_{0,v}^{(r)}\}|}{|\mathcal{I}_k|}. \quad (27)$$

The mean of $\phi_{\text{net},k}^{(r)}$ across runs is used as one of the three decision criteria and is referred to as ϕ_{net} throughout the results. Lower values are preferred, because they mean that a smaller share of directly affected users experiences a travel-time increase.

Two properties of this definition should be kept in mind when reading Chapter 4. First, ϕ_{net} is an unweighted head count: every impacted vehicle contributes equally, regardless of how much time it gained or lost. Second, it is a pure incidence measure and carries no information about magnitude. A plan can therefore delay a large share of the impacted population by a few seconds each and score worse on ϕ_{net} than a plan that delays a slightly smaller share by several minutes each. This is why ϕ_{net} is reported alongside the total additional travel time of Equation (26) rather than instead of it: the two together separate how many users are affected from how badly they are affected.

3.5.6 Multi-criteria decision analysis

The final ranking is built in three steps, following the general MCDA idea of making criteria and weights explicit rather than hiding them in an implicit single score [43]. First, infeasible plans are removed by the completed-trip gate of Section 3.5.4. Second, the remaining plans are screened for Pareto dominance using three cost criteria:

$$(\overline{\Delta D_k}, \overline{\Delta \tau_k}, \overline{\phi_{\text{net},k}}). \quad (28)$$

A feasible plan a Pareto-dominates a feasible plan b if it is no worse on all three criteria and strictly better on at least one. Dominated plans are removed before the weights are applied. This prevents the final ranking from selecting a plan that is worse than another available plan on all measured dimensions.

Third, the plans left on the Pareto front are ranked with a normalized additive score. For each criterion q , values are min-max normalized over the Pareto front:

$$z_{kq} = \frac{x_{kq} - \min_j x_{jq}}{\max_j x_{jq} - \min_j x_{jq}}, \quad (29)$$

with $z_{kq} = 0$ when all surviving plans have the same value on criterion q . The composite score is then

$$S_k = \sum_q w_q z_{kq}, \quad (30)$$

where all criteria are costs and lower scores are better. The reference configuration gives equal weight to network time-loss delay, mean travel time and the fraction of impacted users delayed. This should be read as a neutral starting point, not as a claim that the three dimensions are inherently equally important.

Because the weights contain judgement, the evaluator also checks how sensitive the ranking is to them. It samples weight vectors uniformly from the three-criterion simplex using a Dirichlet(1, 1, 1) distribution. Here a weight vector is a triple (w_1, w_2, w_3) with non-negative entries that sum to one. For each sampled weight vector, the evaluator recomputes the ranking. The report then shows how often each Pareto-efficient plan ranks first, its mean rank, score quantiles and pairwise win probabilities. A plan that wins under the reference weights and also remains strong under many sampled weights is more convincing than a plan that only wins under a narrow weighting assumption.

4 Results

This chapter gives a visual and quantitative readout of the complete pipeline. The first part uses the *city_tiny* test scenario, a toy network generated for this thesis rather than an extract of a real city. It serves as a controlled test bed: the objective is to show that the implemented workflow produces coherent demand, route alternatives, Monte Carlo outputs and a reproducible plan ranking. The third part repeats the same protocol on a small road-only SUMO network around Flagey in Brussels. That second experiment is still synthetic in its demand calibration, but it checks that the pipeline also works on a real neighbourhood topology with irregular streets and OpenStreetMap-derived junction geometry. The final part applies the workflow to a closure of the Delta tunnel in Auderghem, which moves the protocol from an illustrative central edge to an operationally motivated closure of a multi-edge motorway structure and evaluates it under two maintenance regimes: a scheduled 23:00–06:00 night closure and a continuous all-day closure. Figure 12 gives the neutral network view used as the starting point for the first experiment.

city_tiny synthetic network



364 directed edges | 120 junctions | 30.82 lane-km | 20 TAZs

Figure 12: Synthetic `city_tiny` toy network created for this thesis and used as a controlled test case for the experiment. This small network is useful for testing the full pipeline before moving to real street topologies. TAZ boundaries are shown in light grey and the road graph is drawn without closures or detours to give a neutral view of the study area.

4.1 Experimental setup

The scenario generator produced 15,200 vehicle definitions from 24 hourly OD matrices. The closure is directed edge 172, which connects origin junction 2 to destination junction 171. The two manually defined candidate deviation plans also run from junction 2 to junction 171, but they use different replacement corridors. The manually defined plans were evaluated over 30 Monte Carlo runs, while the Yen and upstream batches were evaluated over 10 runs in this experiment. The same evaluator is applied to all plan families, but the number of replications differs at this stage. Figure 13 illustrates the overall pipeline and Table 3 summarises the main results for the manually-defined plans.

Experiment pipeline used for the draft results

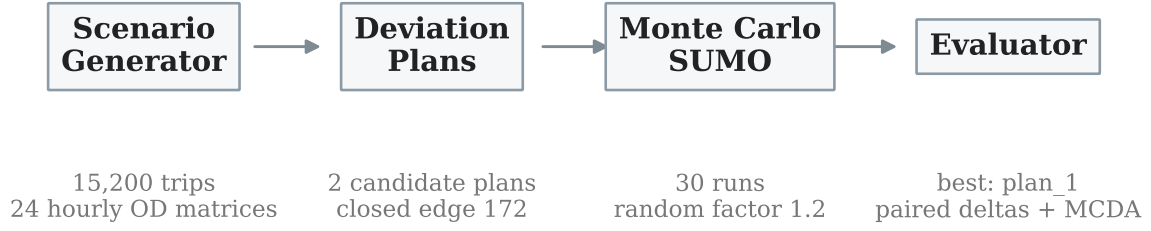


Figure 13: Concrete result-producing pipeline for the experiment. The same generated demand is reused across all Monte Carlo runs, variation comes from stochastic route assignment before each plan-specific simulation.

Plan	Delay	Δ delay	Δ travel	Rerouted
Baseline	63.12	–	–	–
plan_1	69.47	6.36	0.090	1,166.7
plan_2	71.49	8.37	0.151	1,166.7

Table 3: Main Monte Carlo aggregates over 30 runs. Delay and delta delay are in vehicle-hours, delta travel is in minutes per completed vehicle. Deltas are plan minus baseline, where the baseline is the unclosed-network reference.

4.2 Manually-defined plans

The test network contains 364 directed road edges, 364 lanes, 120 junctions and 20 Traffic Analysis Zones (TAZs), for a total of 30.82 lane-kilometres. Figure 14 shows the spatial scale of the experiment. The closed edge sits in the central part of the network and both candidate plans route traffic around this local disruption.

city_tiny network, closure and candidate detours

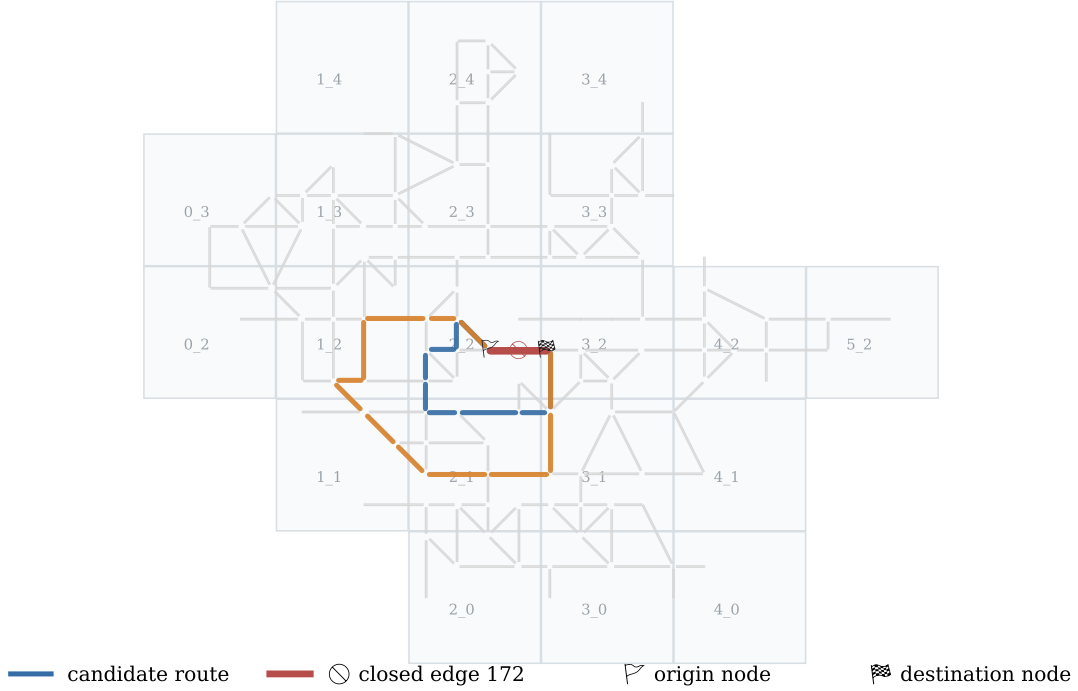


Figure 14: Synthetic network, TAZ grid, closed edge and the two candidate detours. The grey lines are the full road network, red marks the closed edge, blue and orange mark `plan_1` and `plan_2` and the common detour endpoints are origin junction 2 and destination junction 171.

Plan	Origin	Destination	Edges	Length (km)	Interpretation
<code>plan_1</code>	2	171	9	0.635	Short manually defined corridor close to the closure.
<code>plan_2</code>	2	171	12	1.160	Wider manually defined corridor that sends traffic farther around the closure.

Table 4: Explicit origin and destination of the two manually defined candidate plans. Both replace closed edge 172, which itself runs from junction 2 to junction 171.

The generated demand has the intended two-peak structure as illustrated in Figure 15. The largest hourly volume occurs at 17:00, with 1,328 departures, which is consistent with the configured evening-dominant profile. The daily OD matrix is also spatially concentrated rather than uniform: zone 2_1 is the largest origin zone (1,553 trips) and the largest destination zone (1,611 trips) in this synthetic configuration, as suggested by Figure 16.

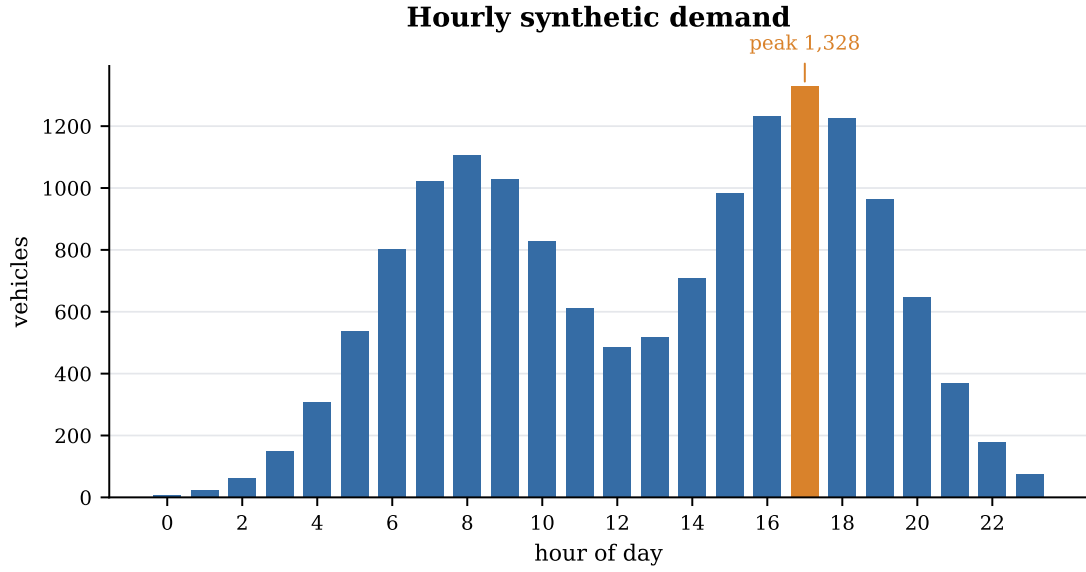


Figure 15: Hourly trip demand generated for the experiment. The evening peak dominates the synthetic day, while off-peak periods remain lightly loaded.

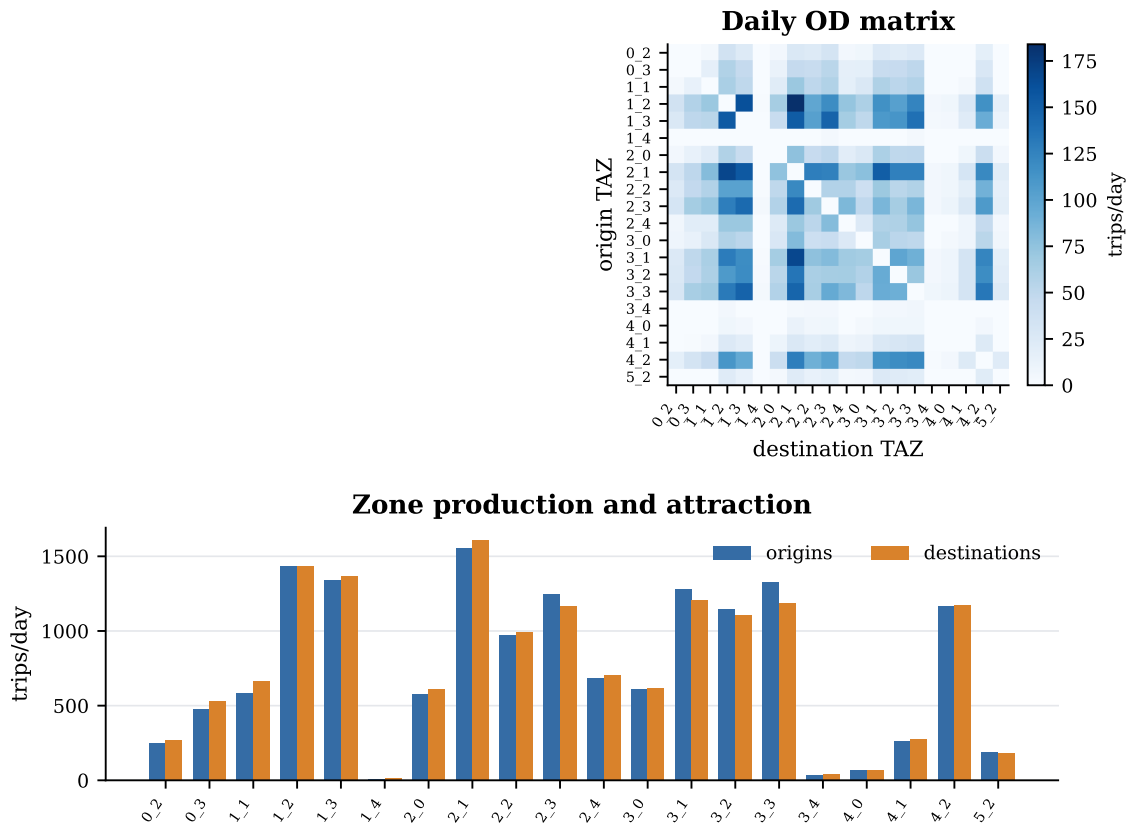


Figure 16: Daily OD structure and zone-level activity. The heatmap gives the full daily OD matrix, while the lower panel compares each zone's generated origins and destinations.

The two tested alternatives differ mainly in path length and spatial spread. `plan_1` uses 9 detour edges, covers 0.635 km and remains closer to the closed link. `plan_2` uses

12 detour edges, covers 1.160 km and makes a wider diversion. In the simulated demand, both alternatives reroute the same average number of vehicles, about 1,167 vehicles per run, corresponding to 7.67% of the daily demand. This makes the comparison easy to read: the difference between plans is driven by the replacement path, not by a different affected-vehicle count.

4.2.1 Evaluator verdict

The evaluator assessed both plans on three criteria aggregated over the 30 Monte Carlo runs: network delay in vehicle-hours (Δ delay), mean added travel time per completed trip (in minutes) and the fraction of impacted users delayed, ϕ_{net} . Both plans are feasible, meaning neither reduces the number of completed trips below the baseline.

plan_1 Pareto-dominates **plan_2** on every criterion: its mean incremental delay is 6.36 vh against 8.37 vh for **plan_2**, the per-vehicle travel-time penalty is +0.090 min against +0.151 min and the fraction of impacted users delayed, ϕ_{net} , is 0.392 against 0.403 as illustrated in Table 5. Because **plan_2** is Pareto-dominated, no weighting of the three criteria can make it the preferred choice. The robustness test confirms this: drawing $N = 20,000$ weight vectors uniformly from the weight simplex (Dirichlet(1, 1, 1) sampling), **plan_1** ranks first in every single draw (100% frequency). The evaluator therefore recommends **plan_1** as the best manually-defined deviation plan for this closure. Figure 17 shows a visual summary of the decision.

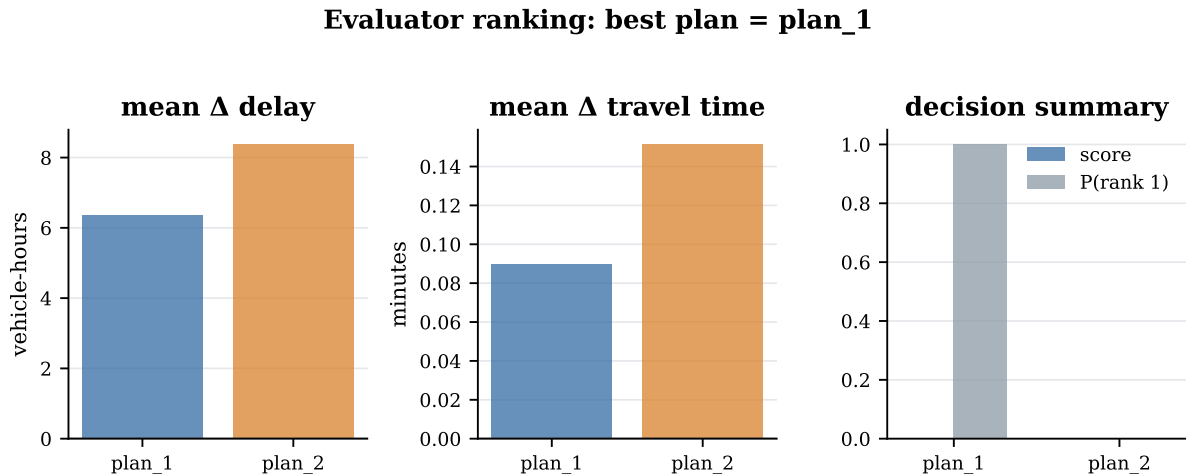


Figure 17: Evaluator decision summary for the two manually-defined candidate plans. **plan_1** Pareto-dominates **plan_2** on all three criteria and is ranked first across all sampled weight vectors. Both alternatives replace closed edge 172 from origin junction 2 to destination junction 171.

4.3 Yen-based plans

While the two candidate plans above were defined manually, the pipeline also offers an automated route-generation mode based on Yen’s k -shortest loopless paths algorithm. Applied to edge 172, the algorithm first resolves the effective detour endpoints. Starting

from the nominal source junction of the closed edge, the graph walks upstream until it finds node 2, the first junction with more than one outgoing edge. It similarly resolves the downstream target to node 171. The closed edge and any approach edges encountered during this endpoint walk are added to the blocked set so they cannot be used as shortcuts.

Yen’s algorithm then enumerates up to k loopless paths from node 2 to node 171 with the blocked set applied. The 8 paths share the same detour origin at node 2 and differ only in the sequence of edges used to reach node 171. They are therefore all depth-0 plans. The two shortest paths (**yen_01** and **yen_02**) each use 8 edges and cover 0.559 km, they are equal-length but traverse distinct edge sequences. Two alternatives use 10 edges and cover 0.627 km, while the remaining four use 9 edges and cover 0.635 km. The longest Yen paths are therefore about 14% longer than the two shortest ones, but still shorter than the wider manual **plan_2**. This is the first place where the automated approach adds value: starting from the same closure endpoints as the expert-defined plans, it exposes two shorter options and several nearby corridor variants. Figure 18 shows the eight alternatives in two panels of at most five routes each, zoomed in on the detour corridor. Within a panel each route has its own colour and is drawn with a small sideways offset, so alternatives that share an edge appear as parallel lines and the small branch differences between equal-length alternatives remain visible.

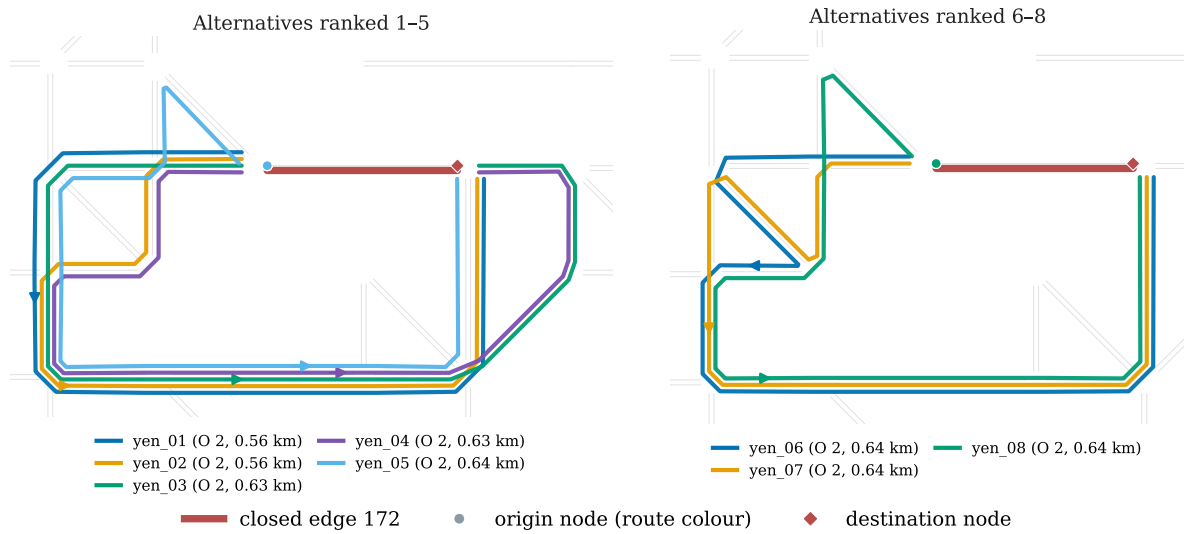


Figure 18: The eight candidate deviation plans produced by Yen’s k -shortest paths algorithm for the closure of edge 172, shown five routes at a time. Each colour is one candidate route, routes sharing an edge are drawn side by side with a small parallel offset. Red marks the closed edge and the per-panel legend gives each route’s origin node and detour length. All Yen alternatives use origin junction 2 and destination junction 171.

The Yen output provides a cost-ordered menu of alternatives all anchored at the immediate source junction of the closure. The two equal-length top paths offer planners a choice of parallel corridors at minimal added distance, while the longer paths trade extra kilometres for greater spatial distribution of rerouted demand.

4.3.1 Evaluator verdict

The evaluator assessed the 8 Yen-generated plans on the same three criteria as before: network delay in vehicle-hours (Δ delay), mean added travel time per completed trip and the fraction of impacted users delayed, ϕ_{net} . All Yen plans are feasible, meaning none reduces the number of completed trips below the baseline.

The Pareto screening leaves only two efficient alternatives: **yen_01** and **yen_06**. **yen_01** gives the lowest delay and travel-time penalty, with a mean incremental delay of 4.35 vh and a mean added travel time of +0.070 min per completed trip. However, **yen_06** has the lowest fraction of impacted users delayed, with $\phi_{\text{net}} = 0.378$ against 0.380 for **yen_01**. This means that **yen_01** is not a strict Pareto winner: it is better on the two time-based criteria, while **yen_06** is slightly better on the congestion criterion.

The remaining six plans are Pareto-dominated. In particular, **yen_01** dominates **yen_02**, **yen_03**, **yen_04**, **yen_05**, **yen_07** and **yen_08**, so these alternatives are removed before weighted scoring. With equal reference weights, the additive score ranks **yen_01** first. The robustness test confirms this preference but not unanimously: drawing $N = 20,000$ weight vectors uniformly from the weight simplex (Dirichlet(1, 1, 1) sampling), **yen_01** ranks first in 74.95% of draws, while **yen_06** ranks first in 25.05% of draws. The evaluator therefore recommends **yen_01** as the best Yen-generated deviation plan for this closure, while identifying **yen_06** as a relevant alternative if the fraction of impacted users delayed is given very high priority.

4.4 Upstream-based plans

The upstream method extends the depth-0 Yen logic by performing a reverse breadth-first search (BFS) from the closure’s source node and running one Dijkstra query to the target node per discovered upstream origin. Each resulting plan is tagged with the BFS depth of its origin, indicating how many junctions upstream of the closure the rerouting begins. A depth- d plan therefore intercepts traffic before it has reached the approach segment, which is relevant when queues propagate backwards under congested conditions.

Applied to edge 172 with a maximum BFS depth of 4, the reverse search discovered 17 reachable origin nodes across depths 0 through 4, each yielding a unique Dijkstra path to destination junction 171, for a total of **17 upstream deviation plans**. The depth distribution is: 1 plan at depth 0 (**upstream_17**, the universal fallback at junction 2), 2 plans at depth 1 (junctions 1 and 12), 4 plans at depth 2 (junctions 6, 8, 22, 28), 4 plans at depth 3 (junctions 15, 157, 159, 165) and 6 plans at depth 4 (junctions 34, 42, 178, 190, 197, 201).

Detour lengths span from 0.351 km (**upstream_01**, depth 4, 4 edges, junction 42) to 0.794 km (**upstream_06**, depth 4, 11 edges, junction 201). Figure 19 then shows the upstream candidates in rank order, at most five routes per panel, following the presentation used for the Yen candidates: each route has its own colour, shared edges are drawn as parallel offset lines and each panel is zoomed to the corridor covered by its routes.

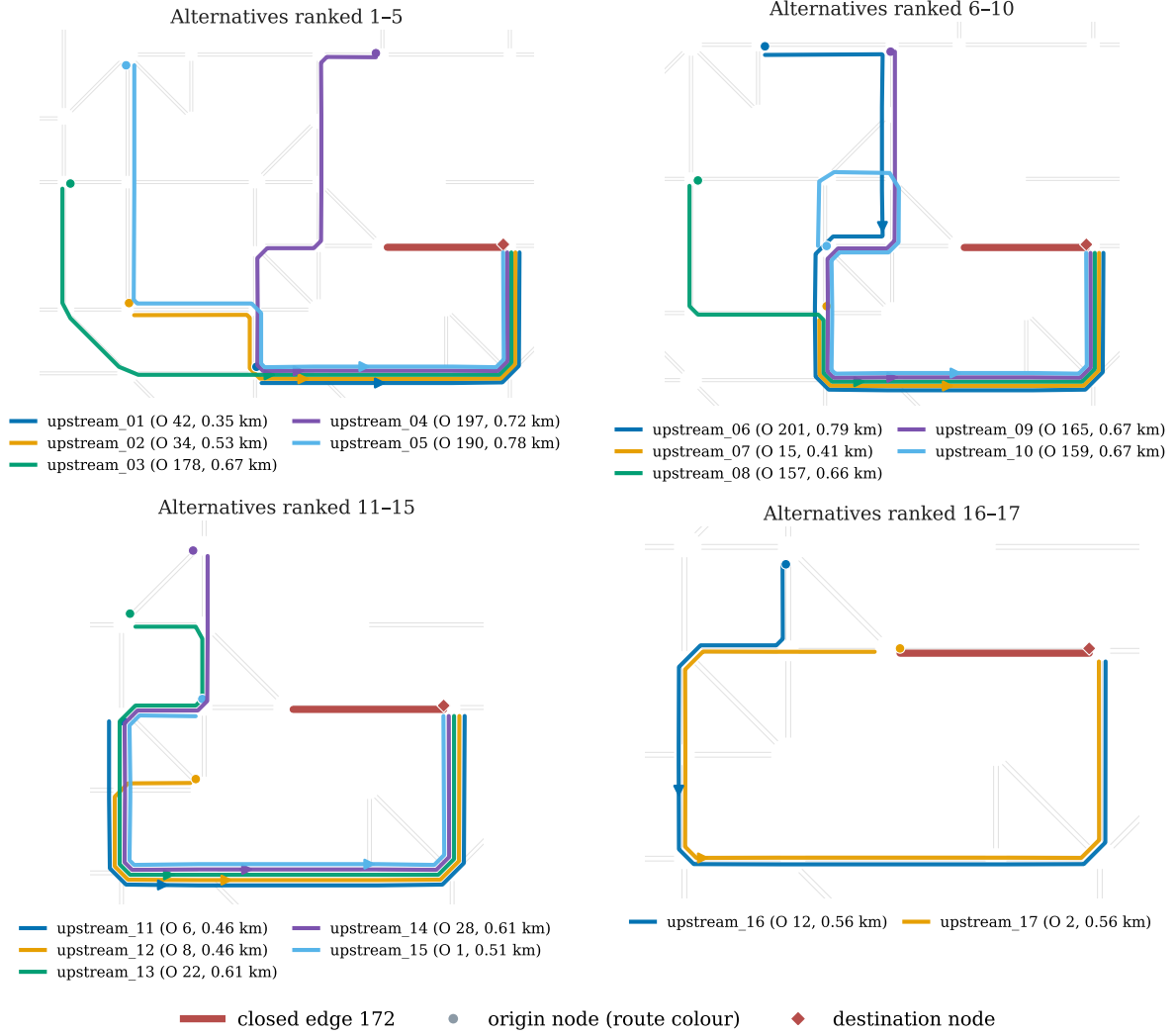


Figure 19: The 17 candidate deviation plans produced by the upstream BFS method for the closure of edge 172, shown five routes at a time in rank order. Each colour is one candidate route, with its origin node marked by a dot in the same colour, routes sharing an edge are drawn side by side with a small parallel offset. The per-panel legend gives each route’s origin node and detour length. All alternatives terminate at destination junction 171, while their origin junctions vary by upstream depth.

Compared with the Yen output, the upstream plans cover a much broader spatial footprint. All Yen paths share the single node 2 origin. In contrast, the upstream set spans 17 distinct interception points before the closure. This is the main practical benefit of the automated approach in this experiment: the manual plans describe two expert-selected corridors from the closure entrance, whereas the upstream method also proposes where traffic can be intercepted earlier if queues begin to propagate. Plans are passed to the route rewriter sorted depth-descending so the most spatially specific plan is tried first, falling back to shallower ones for vehicles that have already passed the deeper diversion points, **upstream_17** (depth 0, junction 2) serves as the guaranteed fallback for all affected vehicles.

4.4.1 Evaluator verdict

The evaluator assessed the 17 upstream-generated plans on the same three criteria: network delay in vehicle-hours (Δ delay), mean added travel time per completed trip and the fraction of impacted users delayed, ϕ_{net} . All upstream plans are feasible, meaning none reduces the number of completed trips below the baseline.

The Pareto screening keeps six efficient alternatives: `upstream_04`, `upstream_08`, `upstream_13`, `upstream_14`, `upstream_15` and `upstream_16`. Among them, `upstream_16` gives the best time-based performance, with the lowest mean incremental delay (3.22 vh) and the lowest mean added travel time (+0.058 min per completed trip). However, it has the highest fraction of impacted users delayed on the Pareto front, with $\phi_{\text{net}} = 0.383$. Conversely, `upstream_15` has the lowest fraction of impacted users delayed ($\phi_{\text{net}} = 0.380$), but a higher delay and travel-time penalty. This creates a genuine multi-criteria trade-off rather than a single plan dominating all others.

The remaining 11 upstream alternatives are Pareto-dominated and are removed before weighted scoring. With equal reference weights, the additive score ranks `upstream_16` first, followed by `upstream_14`, `upstream_04`, `upstream_08`, `upstream_15` and `upstream_13`. The robustness test supports this recommendation: drawing $N = 20,000$ weight vectors uniformly from the weight simplex (Dirichlet(1, 1, 1) sampling), `upstream_16` ranks first in 68.48% of draws. The next most robust first-rank alternative is `upstream_15`, with 19.22%, while `upstream_04` and `upstream_08` rank first in about 6% of draws each. The evaluator therefore recommends `upstream_16` as the best upstream-generated deviation plan for this closure, while noting that the final choice remains sensitive to how strongly the fraction of impacted users delayed is weighted.

4.5 Comparison across generation strategies

Plan	Δ delay (vh)	Δ travel (min)	ϕ_{net}	Runs
<code>plan_1</code>	6.36	0.090	0.392	30
<code>yen_01</code>	4.35	0.070	0.380	10
<code>upstream_16</code>	3.22	0.058	0.383	10

Table 5: Best-ranked plan from each generation strategy. Delta delay is reported in vehicle-hours, delta travel in minutes per completed vehicle and ϕ_{net} as the fraction of impacted users delayed, as defined in Section 3.5.5. The last column gives the number of Monte Carlo replications behind each row: the manual batch was run over 30 seeds and the two automated batches over 10, so the values are not estimated at comparable precision and are reported here for completeness rather than as a controlled comparison.

The three plan families are therefore not compared on delay in this experiment. The manual batch was simulated over 30 Monte Carlo seeds and the Yen and upstream batches over 10, so the incremental-delay means in Table 5 are estimated at different precision and the differences between them cannot be attributed to the generation strategy rather than to the replication budget. Establishing that ranking would require re-running all three

families at a common budget, which is left as future work.

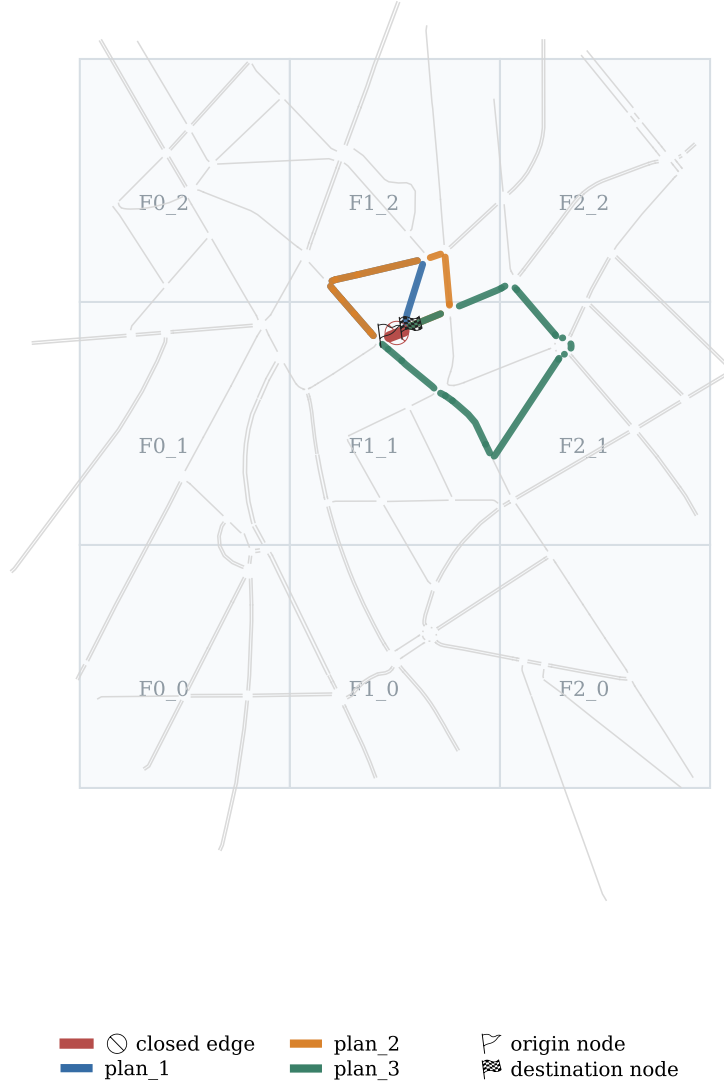
What the experiment does establish is structural and does not depend on the number of replications. Starting from the same closure endpoints as the expert-defined corridors, Yen’s algorithm returned two alternatives of 0.559 km, shorter than either manual plan (0.635 km and 1.160 km), together with six further corridor variants ordered by cost. The upstream search returned 17 distinct plans spread over BFS depths 0 to 4, against the single origin junction 2 available to the manual and Yen plans, so it proposes where traffic can be intercepted before it reaches the closure rather than only how it can be routed around it. Both properties were obtained automatically from the network topology.

This is the practical argument for adding automated generation to the workflow. It does not mean the expert-created plans are poor: they remain in the candidate set and are evaluated on the same terms. It means the candidate set itself is larger and contains alternatives, shorter replacement paths and earlier interception points, that would be tedious to enumerate by hand and that a purely manual process is therefore likely to miss.

4.6 Real-neighbourhood validation

To verify that the workflow is not tied to the regular geometry of the toy network, the same manually-defined-plan protocol was repeated on a small road-only SUMO network around Flagey in Brussels. The network was extracted from OpenStreetMap and filtered to passenger-car roads before conversion to SUMO. It contains 342 directed non-internal road edges, 364 lanes, 201 non-internal junctions and 18.66 lane-kilometres. A simple 3×3 TAZ grid was generated over the largest private-vehicle strongly connected component, producing 9 traffic analysis zones. This zoning is intentionally coarse: it is sufficient for a reproducible pipeline validation, but it is not a calibrated Brussels travel-demand model. Figure 20 shows this extracted topology, the generated TAZ grid, the selected closure and the three candidate detours.

Flagey network, closure and candidate detours



342 directed edges | 201 junctions | 18.66 lane-km | 9 TAZs

Figure 20: Flagey road network used for the real-neighbourhood validation. The grey graph is the road-only SUMO network, the light grid shows the 9 generated TAZs, red marks the closed edge and the three coloured paths show the candidate detours evaluated by Monte Carlo simulation. The common origin and destination are marked as O and D, with the exact node IDs reported in the legend.

The scenario generator produced 2,400 trip requests over one synthetic day. After stochastic routing with `duarouter`, 2,371 vehicles were present and completed in every baseline and plan simulation, the same completed count across all alternatives means the feasibility comparison is not affected by different vehicle losses. The temporal demand profile keeps the same two-peak structure as the toy experiment, with a morning peak

around 08:00 and the largest hourly volume at 17:00. Figure 21 shows the resulting hourly demand profile for this validation scenario.

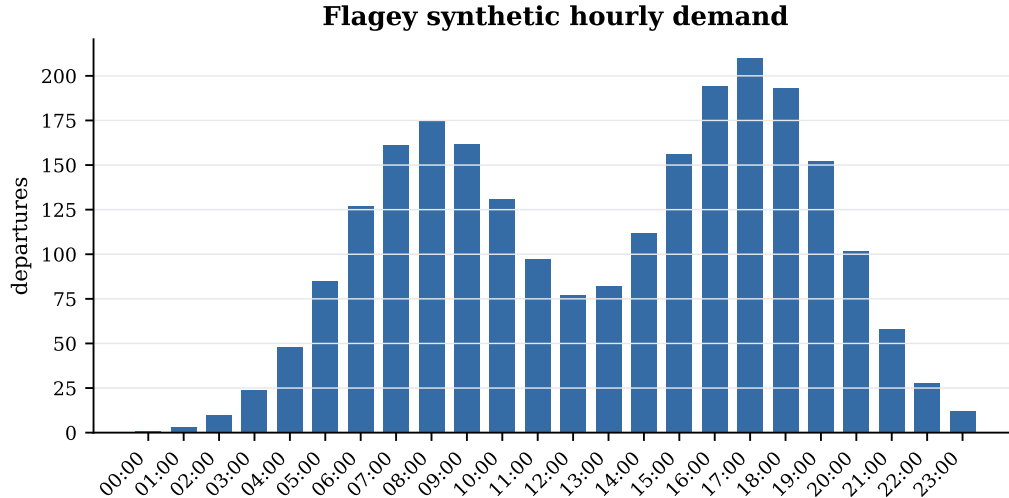


Figure 21: Synthetic hourly demand for the Flagey validation scenario. The profile is generated by the same scenario-generator model as the toy experiment, scaled down to 2,400 daily trip requests for a compact neighbourhood simulation.

The closed road segment is SUMO edge `-414404581#0`. It was selected because it lies in the central part of the extracted neighbourhood and admits several topology-valid source-to-target alternatives once the edge itself is blocked. Three candidate detours were generated from the same immediate closure endpoints: origin node `cluster_207124856_3121275054_4240336304_5121849273_#2` and destination node `cluster_26085294_3121275079_3121275084_3121275100_#1`. They were evaluated over 30 Monte Carlo runs with the same randomised-routing protocol used above. The three alternatives reroute the same average number of vehicles, 233.8 vehicles per run, corresponding to 9.86% of the completed daily demand. The comparison therefore isolates the quality of the replacement path rather than differences in how many vehicles are affected.

Plan	Edges	Length (km)	Delay	Δ delay	Δ travel
Baseline	—	—	27.73	—	—
plan_1	9	0.282	29.07	1.34	0.091
plan_2	12	0.322	28.76	1.03	0.097
plan_3	18	0.546	30.49	2.76	0.203

Table 6: Main Flagey Monte Carlo aggregates over 30 runs. Delay and delta delay are in vehicle-hours, delta travel is in minutes per completed vehicle. Each candidate reroutes the same affected population, 233.8 vehicles per run on average.

4.6.1 Evaluator verdict

The Flagey result is deliberately more nuanced than the two-plan manual result on *city_tiny*. `plan_3` is clearly inferior: it has the largest incremental delay (+2.76 vh), the

largest mean travel-time penalty (+0.203 min per completed trip) and the largest fraction of impacted users delayed ($\phi_{\text{net}} = 0.338$). It is therefore Pareto-dominated by both **plan_1** and **plan_2**.

The choice between **plan_1** and **plan_2** is a genuine multi-criteria trade-off. **plan_2** produces the lowest mean incremental delay, +1.03 vh compared with +1.34 vh for **plan_1**. However, **plan_1** has the lower mean travel-time penalty (+0.091 min compared with +0.097 min) and the lower fraction of impacted users delayed ($\phi_{\text{net}} = 0.309$ compared with 0.327). With equal reference weights across delay, travel time and the fraction of impacted users delayed, the additive MCDA score therefore ranks **plan_1** first. The robustness test supports this recommendation but not unanimously: under 20,000 uniformly sampled weight vectors on the three-criterion simplex, **plan_1** ranks first in 74.51% of draws and **plan_2** in 25.49% of draws. Figure 22 summarises this trade-off and the final evaluator ranking.

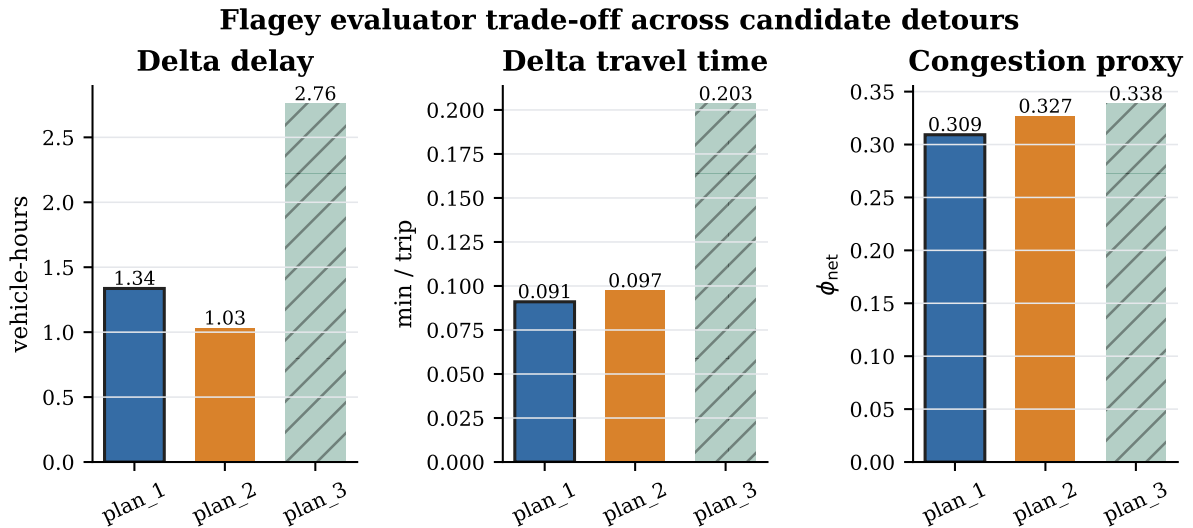


Figure 22: Evaluator trade-off for the Flagey validation experiment. The hatched bars indicate the Pareto-dominated **plan_3**. The selected plan, **plan_1**, is not the minimum-delay alternative, but it gives the best combined result once travel-time and congestion-proxy criteria are included. All three alternatives use the same origin and destination nodes shown in Figure 20.

This validation does not claim calibrated traffic performance for the Flagey area. The demand is synthetic, the TAZ zoning is automatic and the closure is an illustrative central edge chosen to exercise the route-rewriting logic. What it does show is that the thesis pipeline remains operational when the regular toy grid is replaced by a real urban street topology: the scenario generator produces routable demand, the plan representation remains valid, Monte Carlo route variation is reproducible over 30 runs and the evaluator returns an interpretable recommendation with an explicit trade-off rather than a purely visual judgement.

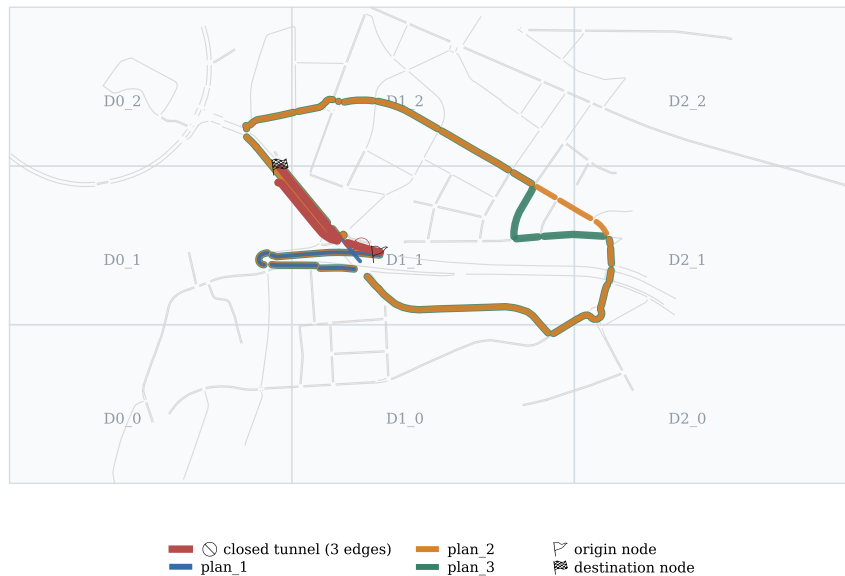
4.7 Closure of the Delta tunnel

The Flagey experiment establishes that the pipeline survives a real street topology. The Delta experiment asks a harder question: whether it can answer an *operationally posed* question, of the kind a road authority actually formulates when it schedules maintenance. The study area is the Delta interchange in Auderghem, where the E411 enters Brussels and the closure is the Delta tunnel between 23:00 and 06:00 [51]. This is precisely the class of decision the present framework is meant to support: a planned, announced closure of a known structure, for which several diversion itineraries exist and must be chosen between and signal timings adjusted, all of it before the first vehicle is affected.

4.7.1 Experimental setup

The network was extracted from OpenStreetMap and filtered to passenger-car roads before conversion to SUMO. It contains 310 directed non-internal road edges, 371 lanes, 163 non-internal junctions and 32.52 lane-kilometres. A 3×3 TAZ grid produces 9 internal zones, to which the external cordon zone is added, for 10 zones in total. Figure 23 shows the extracted topology, the zoning, the three closed carriageways and the three candidate detours.

Delta network, tunnel closure and candidate detours



310 directed edges | 163 junctions | 32.52 lane-km | 9 internal TAZs + 1 cordon zone

Figure 23: Delta road network used for the tunnel night-closure experiment. The grey graph is the road-only SUMO network, the light grid shows the 9 internal TAZs, red marks the three closed tunnel carriageways and the three coloured paths show the candidate detours. The external cordon zone D_EXT is not drawn, since it spans the whole study area rather than a contiguous district. The detours share their first four and last two edges and are drawn with tapering line widths so the shared segments remain readable.

The scenario generator produced 12,000 trip requests over one synthetic day. Figure 24 shows the generated profile with the simulated window highlighted.

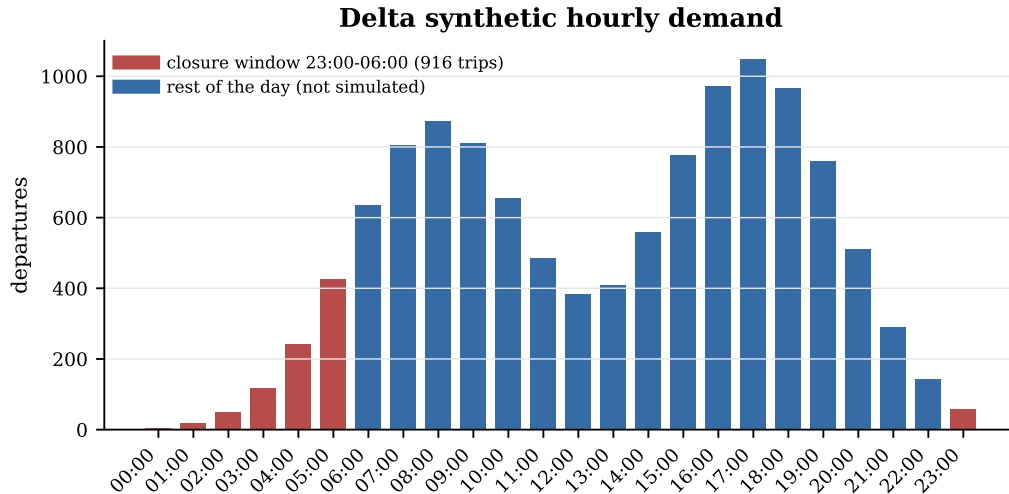


Figure 24: Synthetic hourly demand for the Delta scenario. The profile is produced by the same scenario-generator model as the two earlier experiments. Only the shaded 23:00–06:00 window is simulated, since the closure applies at night, it contains 916 of the 12,000 daily trip requests.

Three candidate detours were generated from the tunnel’s immediate closure endpoints, origin junction on Boulevard du Triomphe where the inbound E411 arrives and destination junction where the tunnel resurfaces. They were evaluated over 30 Monte Carlo runs with the same randomised-routing protocol used above. The three alternatives reroute the same average population, 105.2 vehicles per run, corresponding to 11.49 % of the simulated night demand, so the comparison again isolates the quality of the replacement path rather than differences in the affected-vehicle count.

Plan	Edges	Length (km)	Delay	Δ delay	Δ travel
Baseline	—	—	5.04	—	—
plan_1	7	0.777	6.58	1.54	0.221
plan_2	40	2.610	9.01	3.97	0.861
plan_3	40	2.763	8.88	3.84	0.887

Table 7: Main Delta Monte Carlo aggregates over 30 runs. Delay and delta delay are in vehicle-hours, delta travel is in minutes per completed vehicle. Each candidate reroutes the same affected population, 105.2 vehicles per run on average out of 916 simulated night trips.

4.7.2 Evaluator verdict

All three plans are feasible: none reduces the number of completed trips below the baseline. The paired tests separate them without ambiguity. The incremental delay is +1.54 vh for plan_1, with a 95 % confidence interval of [1.51, 1.57], against +3.97 vh ([3.89, 4.05]) for

plan_2 and +3.84 vh ([3.75, 3.93]) for **plan_3**. All paired t -test and Wilcoxon p -values are below 10^{-6} . Relative to a baseline delay of 5.04 vh, the short bypass adds 31 % while the two routes add roughly 78 %.

The Pareto screening nevertheless retains all three alternatives and the reason is instructive. **plan_1** is best on both time criteria, but it is *worst* on the fraction of impacted users delayed: $\phi_{\text{net}} = 0.909$ against 0.893 for **plan_2** and 0.891 for **plan_3**. Since ϕ_{net} is the share of impacted vehicles whose journey ends up longer than in the baseline, this says that the short bypass inconveniences a slightly larger fraction of the tunnel’s users while costing each of them far less time: over the impacted population of 110 vehicles, **plan_1** adds 180 vehicle-minutes in total against 705 and 727 for the two routes. No alternative dominates on all three criteria, so none can be screened out before weighting.

With equal reference weights the additive score ranks **plan_1** first. The robustness test supports the recommendation without making it unanimous: under $N = 20,000$ weight vectors drawn uniformly from the three-criterion simplex (Dirichlet(1, 1, 1) sampling), **plan_1** ranks first in 74.20 % of draws and **plan_3** in 25.80 %, while **plan_2** never ranks first. The evaluator therefore recommends **plan_1**, the 0.777 km bypass, while identifying **plan_3** as the relevant alternative if the fraction of impacted users delayed is given very high priority. Figure 25 summarises the trade-off.

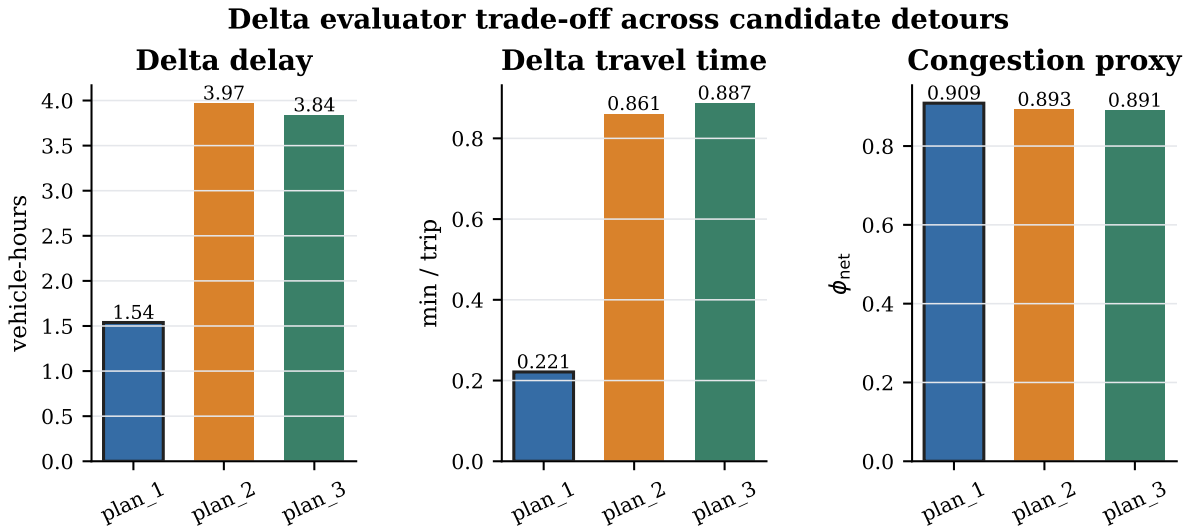


Figure 25: Evaluator trade-off for the Delta tunnel night closure. No candidate is Pareto-dominated: **plan_1** minimises both time-based criteria but has the highest fraction of impacted users delayed, since it slows a marginally larger share of the impacted vehicles while costing each of them substantially less time. All three alternatives use the origin and destination nodes shown in Figure 23.

The near-identical robustness split at Flagey (74.51 % versus 25.49 %) and at Delta (74.20 % versus 25.80 %) is a coincidence of the two scenarios rather than a property of the method, but it does illustrate a recurring pattern in these experiments: the plan that minimises aggregate delay is usually not the plan that spreads the burden most evenly and the MCDA layer is what makes that tension explicit instead of hiding it behind a single headline number.

4.7.3 Night closure versus all-day closure

A night closure is the mild case. A road authority choosing between maintenance regimes will also want to know what the same three plans cost if the tunnel has to stay shut around the clock, for instance because the works cannot be demobilised every morning. Because the closure is a property of the plan file and not of the demand, that variant requires changing only the simulated window: the identical network, the identical three plans and the identical generated demand were re-run over the full day, so the 12,000 trip requests are exposed to the closure instead of the 916 that depart during the night window.

The demand exposed to the closure grows by a factor of 13.10. If congestion played no role, every plan’s delay penalty would grow by the same factor. It does not and that is the substantive result of the comparison. Table 8 reports both regimes side by side and Figure 26 shows the same information graphically.

Plan	Δ delay (vh)		Δ travel (min)		Delay growth
	night	all day	night	all day	
plan_1	1.54	20.92	0.221	0.221	13.6×
plan_2	3.97	193.81	0.861	1.548	48.8×
plan_3	3.84	200.17	0.887	1.613	52.1×

Table 8: The same closure and the same three plans under the two exposure regimes, 30 Monte Carlo runs each. The night window contains 916 trips and the full day 12,000, a demand ratio of 13.10. The last column is the ratio of the all-day to the night-time incremental delay, a plan that scaled purely with demand would show 13.1×

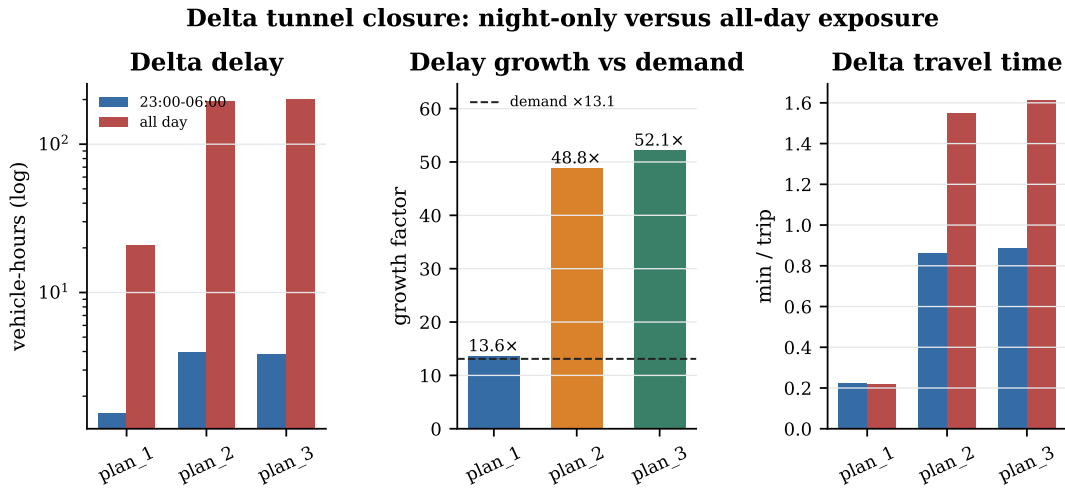


Figure 26: Delta tunnel closure under night-only and all-day exposure. The left panel uses a logarithmic axis because the two regimes differ by two orders of magnitude. The centre panel divides each plan’s all-day delay penalty by its night-time penalty, the dashed line is the demand ratio of 13.1×, so bars above it indicate a penalty growing faster than the traffic that causes it. The right panel shows that the per-vehicle cost of plan_1 is unchanged between regimes while the two routes almost double.

plan_1 scales at $13.6\times$, essentially in proportion to demand: its per-vehicle travel-time penalty is 0.221 min at night and 0.221 min across the full day, unchanged to three decimals. The 0.777 km bypass is short enough and its corridor spare enough that it never becomes the binding constraint, so its cost is close to a fixed toll per rerouted vehicle. The two routes behave differently: **plan_2** scales at $48.8\times$ and **plan_3** at $52.1\times$, roughly four times faster than demand and their per-vehicle penalties rise from 0.861 to 1.548 min and from 0.887 to 1.613 min respectively. Sending 1,330 vehicles per run around a 2.6 km loop through residential streets saturates that loop and the vehicles then interact with each other rather than simply travelling further.

The practical consequence is a change in the *stakes* rather than in the recommendation. In both regimes the evaluator returns **plan_1** and it does so with almost identical robustness: first in 74.20 % of weight draws at night and 74.95 % across the full day, with **plan_3** taking the remainder in both cases. What changes is the cost of getting the choice wrong. At night, picking a route instead of the bypass costs an extra 2.3 to 2.4 vh of delay, across the full day it costs an extra 173 to 179 vh and the ratio between the worst and best candidate widens from 2.6 to 9.6. A deviation plan that looks merely suboptimal under light demand can be an operational failure under full demand, which is an argument for evaluating candidates at the loading they will actually experience rather than at a convenient one.

Two secondary effects point the same way. First, **plan_2** loses its feasibility: in one of the 30 all-day runs it fails to complete a vehicle that the baseline completes (11,999 against 12,000) and the evaluator therefore rejects it before weighted scoring, leaving a Pareto front of **plan_1** and **plan_3**. At night all three plans were feasible. The throughput constraint only becomes informative once the network is loaded enough for a plan to actually fail. Second, the Monte Carlo precision moves in opposite directions for the two plan families. The 95 % confidence half-width of **plan_1**'s delay penalty falls from 1.91 % to 0.54 % of the mean, because more vehicles average out the routing noise, whereas **plan_2** and **plan_3** degrade from 2.02 % and 2.34 % to 7.33 % and 6.59 %. Once a corridor is congested, small differences in stochastic route assignment decide whether it tips into queueing, so the replication budget needed for a stable answer is itself regime-dependent. The 30 runs used here are ample for the night comparison and adequate but no longer generous for the all-day one.

The fraction of impacted users delayed tells the complementary story. Figure 27 shows the all-day trade-off in the same layout used for the night closure. Under full-day loading, ϕ_{net} rises for **plan_1** from 0.909 to 0.958 while it stays near 0.908 for the two routes, so the bypass now slows almost every one of the 1,327 impacted vehicles. It nevertheless remains the recommended plan by a wide margin on the time criteria: over the impacted population it adds 2,302 vehicle-minutes against 14,201 and 14,819 for the routes. The trade-off that made the night-time decision genuinely multi-criteria is still present, but it is now clearly subordinate to the delay difference.

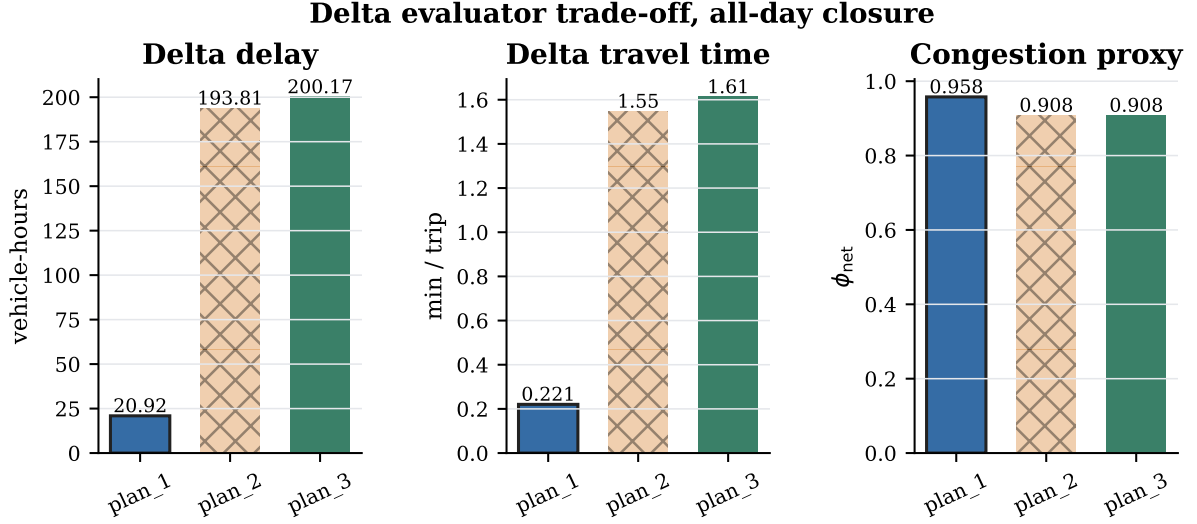


Figure 27: Evaluator trade-off for the all-day Delta tunnel closure, in the same layout as Figure 25. The cross-hatched bars mark **plan_2**, which is rejected on feasibility because one run fails to complete a vehicle the baseline completes. **plan_1** remains the recommended plan and its advantage on the delay criterion is an order of magnitude larger than in the night-only regime.

As with the Flagey validation, this experiment does not claim calibrated traffic performance for the Delta interchange. The demand is synthetic, the TAZ zoning automatic and the 12,000 daily trip requests are modelling assumptions rather than observed quantities, they are single constants in the configuration and are the natural targets of a sensitivity analysis. Two structural caveats also follow from the extract itself. The tunnel is inbound-only within the bounding box, so what is measured is the cost of closing an *entry* corridor rather than a fully bidirectional one and a wider extract would be required to capture the surface return movement. And because junction 8792776 offers a single alternative exit, the candidate set is unusually thin, a closure elsewhere in the same network would give the plan generators more room. What the experiment does show is that the workflow extends from an illustrative single-edge closure to a scheduled, multi-edge, time-limited closure of the kind that motivates the tool in practice and that it returns a defensible recommendation together with the trade-off that qualifies it. The night-versus-all-day comparison adds a further point that the two earlier experiments could not make: because the closure window is a parameter of the simulation rather than of the demand or the plans, the same candidate set can be re-scored under a different maintenance regime at the cost of one more Monte Carlo batch and the answer to *which plan* can be stable while the answer to *how much it matters* is not.

5 Discussion

5.1 Usefulness of the framework

The main value of the work is that it turns deviation planning from a manually inspected routing problem into a reproducible evaluation workflow. Starting from a SUMO network, a TAZ layer and a small set of configuration parameters, the framework produces synthetic demand, generates candidate detours, rewrites affected vehicle routes, runs paired simulations and ranks the resulting plans with the same criteria. This is useful even when the absolute numerical results are not yet calibrated to a real city, because it provides a controlled environment in which different diversion strategies can be compared on equal terms.

The experiment also shows that automated candidate generation can add information beyond the manually selected detours. In the *city_tiny* case, the best Yen-generated plan reduces incremental delay relative to the best manual plan, while the upstream method finds earlier interception points that are not visible when the detour is defined only from the source of the closed edge. This does not replace expert judgement: a traffic manager still needs to check local constraints, signing possibilities and policy priorities. However, the automated methods widen the candidate set and make it easier to notice alternatives that would otherwise be tedious to enumerate by hand.

A second useful aspect is the paired Monte Carlo evaluation. Each candidate plan is compared with the matching baseline under the same demand and routing seed, so the reported deltas describe the effect of the plan rather than an uncontrolled difference between simulation runs. The stochastic repetitions also expose whether the recommendation is robust. For the manual plans, the ranking is unambiguous because `plan_1` dominates `plan_2`. For the Yen and upstream candidates, the results show a more realistic decision problem: the best time-based plan is not always the best plan on the fraction of impacted users delayed, so the preferred option depends on how the decision maker weights the criteria.

This is an important implication of using a Pareto-based decision layer. When several candidates remain non-dominated, the evaluation should not be read as having failed to identify the best deviation plan. Rather, it reveals that the alternatives embody different compromises. One plan may minimise total network delay while increasing the burden on the vehicles that are directly affected by the closure; another may be less efficient on average but avoid severe spillback, distribute traffic more evenly across the network or perform more robustly across stochastic runs. The final choice is therefore partly a normative and operational decision. Besides mean delay and travel time, a traffic authority may need to evaluate the spatial distribution of congestion, the equity of the burden placed on different origin-destination pairs or neighbourhoods, the variance and worst-case outcomes across Monte Carlo runs, the sensitivity of the ranking to MCDA weights, and the practical feasibility of communicating and enforcing the plan on the ground. In such cases, the framework is most useful when it exposes the trade-off surface and identifies a set of defensible alternatives, rather than forcing all candidates into a single apparently

objective ranking.

The decision layer is intentionally simple, but that simplicity is useful for operational interpretation. Feasibility filtering, Pareto screening and weight-sensitivity analysis make the recommendation traceable: a rejected plan can be rejected because it is infeasible, because another plan dominates it, or because it only wins under a narrow set of weights. The framework therefore acts less as an automatic prescription tool and more as a structured decision-support tool that documents why a deviation plan is preferred.

5.2 Limitations

These contributions remain bounded by several methodological and modelling choices. The following limitations are grouped by the stage of the pipeline they affect and are discussed in terms of the threat they pose to internal and external validity.

5.2.1 Single-closure assumption

The deviation plan maker and the route rewriter are designed around a single closure at a time. The closure itself may span several edges: the Delta experiment blocks a contiguous set of three carriageways and the endpoint computation extends that set further upstream and downstream whenever the immediate junctions admit no diverging movement, so a worksite covering a corridor or a whole junction is already within scope. What the framework does not support is more than one closure being active simultaneously in different parts of the network, which is common when several worksites are scheduled in the same period or when an incident closes a link while planned works are already under way. Running the pipeline twice for two disjoint closures produces two plans that are each optimal in isolation but may conflict when applied together, for instance because both divert traffic onto the same corridor or because the detour of one closure passes through the other. Supporting this case requires more than enlarging the blocked-edge set: the candidate generation would have to reason about a blocked set that is itself the union of several disruptions, and the evaluation would have to attribute the resulting delay to the interaction between the plans rather than to either of them alone. That attribution problem, rather than the implementation, is what needs to be studied first.

5.2.2 No intra-zone demand

Demand is synthesised from TAZ-level OD matrices, which by construction assign all trip origins and destinations to zone centroids. Trips whose true origin and destination both lie inside the same TAZ are therefore excluded from the demand entirely. In the *city_tiny* scenario each TAZ is small and the synthetic demand is deliberately cross-zonal, so this simplification has limited impact, however, on real networks with large zones, intra-zonal demand can represent a significant share of daily trips. Vehicles that depart and arrive within the same zone are also the most likely to use short local roads in the vicinity of the closed edge, meaning that the framework may underestimate the fraction of demand directly affected by the closure.

5.2.3 Hourly OD time resolution

OD matrices are aggregated over one-hour intervals. Sub-hourly demand surges (such as the sharp traffic spike at the end of a sports event or the fifteen-minute burst following a school bell) are smoothed away. Because duarouter distributes departures uniformly within each hourly bucket, the simulated traffic loading during the closure is an average over the full hour rather than a realistic second-by-second profile. This can cause the MC evaluation to underestimate peak-period congestion on detour corridors, which in turn biases the incremental delay estimates downward for high-density time windows.

5.2.4 Demand fixed across Monte Carlo runs

Stochasticity in the Monte Carlo loop enters exclusively through duarouter’s randomised edge weights. The vehicle population (departure times, origin-destination pairs) is generated once and held constant across all 30 runs. Real-world demand is itself uncertain: travellers receive information about the closure at different times, some cancel their trips and others shift their departure time. Demand-side uncertainty is therefore not captured and the variance in the MC outputs reflects only route-assignment variability, not the full uncertainty of real closure impacts.

5.2.5 Single vehicle class

The simulation models only private passenger cars. Freight vehicles, buses, trams, cyclists and pedestrians are absent from the demand. This matters particularly for plan evaluation: heavy goods vehicles have different routing constraints (height and weight restrictions, turning radii) that may make some candidate detour edges physically infeasible for them even if they are optimal for private cars. Emergency services are similarly excluded, so the impact of a closure on response times is not assessed.

5.2.6 Static plan generation

Plans are computed once from the network topology at the moment the closure is declared. The framework does not adapt plans to the real-time traffic state: if a candidate detour is already near saturation before the closure occurs, the plan generator has no mechanism to avoid it. A natural extension would integrate a live traffic data feed so that edge weights at plan generation time reflect current conditions rather than free-flow speeds.

6 Conclusion

This thesis started from a question that road authorities face routinely and answer informally: *when a road segment must be closed, which of the possible detours should be signposted?* The usual answer relies on local expertise and on a visual reading of the network. The objective here was not to replace that expertise, but to give it a reproducible experimental basis, a workflow in which candidate deviation plans are generated systematically, simulated under identical conditions, compared with paired statistics and ranked according to criteria that are stated explicitly rather than left implicit.

What was built

The result is an end-to-end pipeline built around SUMO. A scenario generator synthesises network-aware demand from a road network and a Traffic Analysis Zone layer, combining a gravity origin–destination model, a configurable temporal profile and a k -shortest route library into routable trip definitions. A deviation plan maker operates on a junction-level graph representation of the network and produces candidate detours by three strategies: manual specification, Yen’s k -shortest loopless paths between the closure endpoints, and an upstream search that moves the interception point backwards through the network before searching, so that traffic can be diverted earlier than the closed edge itself. A route rewriter applies each plan to the vehicles whose routes traverse the closure. A Monte Carlo layer then executes repeated simulations in which randomised edge weights perturb route assignment, pairing each plan run with a baseline run generated under the same seed so that the reported deltas measure the effect of the plan rather than an uncontrolled difference between simulations. Finally, an evaluator rejects plans failing a throughput feasibility gate, screens the survivors for Pareto dominance across incremental delay, mean added travel time and the fraction of impacted users delayed and ranks the non-dominated set with additive weighted scoring whose sensitivity to the chosen weights is quantified by resampling the weight simplex.

What was established

Four findings follow from the experiments.

First, automated candidate generation adds information that manual selection misses. On the *city_tiny* network, Yen’s algorithm returned replacement paths shorter than either manually defined corridor while starting from the same closure endpoints and the upstream search returned seventeen distinct interception points spread over four levels of upstream depth, against the single origin junction available to the manual and Yen plans. Those earlier interception points are simply not visible when detours are defined only from the source junction of the closed edge. Both automated families also achieved lower mean incremental delay than the best manual plan in these runs, although the plan families were not simulated with the same number of Monte Carlo replications, so that particular

comparison is indicative rather than established.

Second, the pipeline is not tied to the regular geometry of a generated test network. Repeating the protocol on an OpenStreetMap extract around Flagey and then on the multi-edge Delta tunnel closure in Auderghem, produced routable demand, valid plan representations, reproducible Monte Carlo variation and an interpretable recommendation in every case. The Delta case also demonstrates that the workflow extends from an illustrative single-edge closure to a scheduled, time-limited closure of the kind that motivates the tool in practice.

Third, the decision problem is genuinely multi-criteria and treating it as single-objective would have hidden that. In both real topologies the plan minimising aggregate delay was not the plan distributing the burden most evenly across the impacted population and the recommended plan ranked first in only about three quarters of sampled weight vectors. At Delta the short bypass inconveniences a marginally larger share of tunnel users while costing each of them far less time (180 against roughly 700 vehicle-minutes over the impacted population). Exposing that tension is more useful to a decision maker than resolving it arbitrarily.

Fourth and most strongly, the exposure regime changes the stakes of the decision even when it does not change the decision. Re-running the identical Delta network, plans and demand over a full day instead of the 23:00–06:00 window increased the exposed demand by a factor of 13.1, but the delay penalty of the two routes grew by factors of 48.8 and 52.1 while the short bypass grew by 13.6, essentially in proportion to demand. The recommendation was unchanged, yet the cost of choosing wrongly rose from about 2.3 to about 175 vehicle-hours and the worst-to-best ratio widened from 2.6 to 9.6, one plan additionally lost its feasibility. A plan that looks merely suboptimal under light demand can be an operational failure under full demand, which argues for evaluating candidates at the loading they will actually experience rather than at a convenient one.

What was not established

The framework’s correctness was demonstrated, not its predictive accuracy. All demand is synthetic: origin–destination flows come from a gravity model with assumed parameters, zoning is an automatic grid rather than a calibrated partition and the external-traffic share and daily trip totals are configuration constants rather than observed quantities. No result presented here should be read as a forecast of delay for Flagey or Delta. The pipeline models a single vehicle class, excludes intra-zonal trips, resolves demand at hourly granularity, holds the vehicle population fixed across Monte Carlo runs so that only route-assignment uncertainty is captured and generates plans statically from free-flow topology at the moment the closure is declared. These choices are discussed in Section 5.2 and bound the external validity of every number reported.

Perspectives

Three extensions would be the most valuable to prioritise. The first is validation against an observed closure: comparing simulated detour flows with counting-device measurements

collected during a real roadworks event is the only way to test whether the ranking the framework produces matches what happens on the road. The second is demand-side uncertainty. Because stochasticity currently enters only through randomised route assignment, the reported confidence intervals understate the true uncertainty of a closure's impact, resampling departure times, trip cancellations and information-diffusion delays across Monte Carlo runs would reflect the decision risk more honestly. The third is relaxing the single-closure assumption, so that contiguous worksites and concurrent closures can be planned jointly rather than in isolation, since plans that are optimal separately may conflict when applied together. Beyond these, feeding live traffic states into plan generation and adding equity and spatial-distribution criteria to the evaluator would move the framework closer to the operational role it is designed to serve.

Even in its present form, however, it makes the case for a modest but concrete shift in practice: deviation plans are comparable objects and a city that generates them systematically, simulates them under identical conditions and documents why one was preferred has a defensible answer where it previously had only an informed opinion.

Disclosure of AI Tools

AI-assisted tools were used for language, formatting, LaTeX hygiene and software-development support during the preparation of this work. The author remains responsible for the technical choices, analysis, results and conclusions.

References

- [1] Digital Twin consortium. *Definition of a digital twin*. 2025. URL: <https://www.digitaltwinconsortium.org/initiatives/the-definition-of-a-digital-twin/> (visited on 04/28/2025).
- [2] IBM. *What is a digital twin?* 2021. URL: <https://www.ibm.com/think/topics/what-is-a-digital-twin> (visited on 04/28/2025).
- [3] Evangelia Katsoulakis et al. “Digital twins for health: a scoping review”. In: *npj Digital Medicine* 7.1 (2024), p. 77.
- [4] David Jones et al. “Characterising the Digital Twin: A systematic literature review”. In: *CIRP Journal of Manufacturing Science and Technology* 29 (2020), pp. 36–52. ISSN: 1755-5817. DOI: <https://doi.org/10.1016/j.cirpj.2020.02.002>. URL: <https://www.sciencedirect.com/science/article/pii/S1755581720300110>.
- [5] Axel Schaffland, Jonas Nelson, and Julius Schöning. “Simulating Traffic Networks: Driving SUMO Towards Digital Twins”. In: *SUMO Conference Proceedings* 5 (July 2024), pp. 113–125. DOI: 10.52825/scp.v5i.1105. URL: <https://www.tib-op.org/ojs/index.php/scp/article/view/1105>.
- [6] Sheida Hadavi et al. “Analyzing passenger and freight vehicle movements from automatic-Number plate recognition camera data”. English. In: *European Transport Research Review* 12.1 (May 2020), pp. 1–17. ISSN: 1867-0717. DOI: 10.1186/s12544-020-00405-x.
- [7] Arianna Anniciello et al. “Digital Twins for Traffic Congestion in Smart Cities: A Novel Solution Using Data Mining Techniques.” In: *KMIS*. 2023, pp. 241–248.
- [8] Andrey Rudskoy, Igor Ilin, and Andrey Prokhorov. “Digital twins in the intelligent transport systems”. In: *Transportation Research Procedia* 54 (2021), pp. 927–935.
- [9] Krešimir Kušić, René Schumann, and Edouard Ivanjko. “A digital twin in transportation: Real-time synergy of traffic data streams and simulation for virtualizing motorway dynamics”. In: *Advanced Engineering Informatics* 55 (2023), p. 101858. ISSN: 1474-0346. DOI: <https://doi.org/10.1016/j.aei.2022.101858>. URL: <https://www.sciencedirect.com/science/article/pii/S1474034622003160>.
- [10] Li Deren, Yu Wenbo, and Shao Zhenfeng. “Smart city based on digital twins”. In: *Computational Urban Science* 1.1 (Mar. 29, 2021), p. 4. ISSN: 2730-6852. DOI: 10.1007/s43762-021-00005-y. URL: <https://doi.org/10.1007/s43762-021-00005-y>.
- [11] Andrea Guastella Davide et al. *A Simulation Tool to Assess the Impact of Deviation Plans on Disruptive Events of Urban Traffic*.
- [12] Talha Azfar and Ruimin Ke. *Traffic Co-Simulation Framework Empowered by Infrastructure Camera Sensing and Reinforcement Learning*. 2024. arXiv: 2412.03925 [eess.SY]. URL: <https://arxiv.org/abs/2412.03925>.
- [13] Cláudio Gomes et al. “Co-simulation: a survey”. In: *ACM Computing Surveys (CSUR)* 51.3 (2018), pp. 1–33.

- [14] Mehdi A.Kamran et al. “Traffic Light Signal Timing Using Simulation”. In: *Communications on Advanced Computational Science with Applications* 2017 (Jan. 2017), pp. 1–11. DOI: 10.5899/2017/cacsa-00061.
- [15] Devansh Jalota et al. *Online Traffic Routing: Deterministic Limits and Data-driven Enhancements*. 2021. DOI: 10.48550/arXiv.2109.08706. arXiv: 2109.08706 [math.OC]. URL: <https://arxiv.org/abs/2109.08706>.
- [16] Tommaso Toso, Alain Y. Kibangou, and Paolo Frasca. *Potential detrimental effects of real-time route recommendations in traffic networks*. 2023. DOI: 10.48550/arXiv.2303.17923. arXiv: 2303.17923 [eess.SY]. URL: <https://arxiv.org/abs/2303.17923>.
- [17] J. de Dios Ortúzar and L.G. Willumsen. *Modelling Transport*. Wiley, 2011. ISBN: 9781119993520. URL: <https://books.google.be/books?id=qWa5MyS4CiwC>.
- [18] Davide Andrea Guastella et al. *Traffic Modeling with SUMO: a Tutorial*. 2025. arXiv: 2304.05982 [cs.NI]. URL: <https://arxiv.org/abs/2304.05982>.
- [19] Davide Andrea Guastella, Bruno Cornelis, and Gianluca Bontempi. “Traffic Simulation with Incomplete Data: the Case of Brussels”. In: *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Methods for Enriched Mobility Data: Emerging Issues and Ethical Perspectives 2023*. EMODE ’23. Hamburg, Germany: Association for Computing Machinery, 2023, pp. 15–24. DOI: 10.1145/3615885.3628004. URL: <https://doi.org/10.1145/3615885.3628004>.
- [20] Nimrod Serok, Shlomo Havlin, and Efrat Blumenfeld Lieberthal. “Identification, cost evaluation, and prioritization of urban traffic congestions and their origin”. In: *Scientific Reports* 12.1 (2022), p. 13026.
- [21] Sertan Çolak et al. “Understanding congested travel in urban areas”. In: *Nature Communications* 7 (2016), p. 10793. DOI: 10.1038/ncomms10793.
- [22] Krajzewicz Daniel et al. “Recent Development and Applications of SUMO – Simulation of Urban MObility”. In: *International Journal on Advances in Systems and Measurements* 5 (2012), pp. 128–138. URL: https://sumo.dlr.de/pdf/sysmea_v5_n34_2012_4.pdf.
- [23] Gregor Laemmel. *Simulation of Urban MObility - SUMO*. 2025. URL: https://www.eclipse.org/community/eclipse_newsletter/2017/august/article2.php (visited on 04/29/2025).
- [24] Wagner Peter and other. *Simulation of Urban MObility*. 2025. URL: <https://eclipse.dev/sumo/> (visited on 04/29/2025).
- [25] German Aerospace Center (DLR) and others. *SUMO at a Glance*. 2025. URL: https://sumo.dlr.de/docs/SUMO_at_a_Glance.html (visited on 04/29/2025).
- [26] German Aerospace Center (DLR) and others. *Scenarios*. 2025. URL: <https://sumo.dlr.de/docs/Data/Scenarios.html> (visited on 04/29/2025).
- [27] Karl Schrab, Robert Protzmann, and Ilja Radusch. “A Large-Scale Traffic Scenario of Berlin for Evaluating Smart Mobility Applications”. In: *Smart Energy for Smart Transport*. Ed. by Eftihia G. Nathanail, Nikolaos Gavanias, and Giannis Adamos. Cham: Springer Nature Switzerland, 2023, pp. 276–287.

- [28] Maxime Gueriau and Ivana Dusparic. “Quantifying the impact of connected and autonomous vehicles on traffic efficiency and safety in mixed traffic”. In: *23rd IEEE International Conference on Intelligent Transportation Systems*. 2020.
- [29] Marco Rapelli, Claudio Casetti, and Giandomenico Gagliardi. “Vehicular Traffic Simulation in the City of Turin from Raw Data”. In: *IEEE Transactions on Mobile Computing* (2021), pp. 1–12. DOI: 10.1109/TMC.2021.3075985.
- [30] Ozan K. Tonguz. *How Vehicle-to-Vehicle Communication Could Replace Traffic Lights and Shorten Commutes*. 2018. URL: <https://spectrum.ieee.org/how-vehicle-to-vehicle-communication-could-replace-traffic-lights-and-shorten-commutes> (visited on 04/29/2025).
- [31] Chiara Bachechi, Laura Po, and Federica Rollo. “Big Data Analytics and Visualization in Traffic Monitoring”. In: *Big Data Research* 27 (2022), p. 100292. ISSN: 2214-5796. DOI: <https://doi.org/10.1016/j.bdr.2021.100292>. URL: <https://www.sciencedirect.com/science/article/pii/S221457962100109X>.
- [32] German Aerospace Center (DLR) and others. *Randomness*. 2025. URL: <https://sumo.dlr.de/docs/Simulation/Randomness.html> (visited on 04/23/2026).
- [33] Longchao Da et al. *CityFlowER: An Efficient and Realistic Traffic Simulator with Embedded Machine Learning Models*. 2024. arXiv: 2402.06127 [cs.MA]. URL: <https://arxiv.org/abs/2402.06127>.
- [34] Huichu Zhang et al. “CityFlow: A Multi-Agent Reinforcement Learning Environment for Large Scale City Traffic Scenario”. In: *The World Wide Web Conference. WWW ’19*. ACM, May 2019, pp. 3620–3624. DOI: 10.1145/3308558.3314139. URL: <http://dx.doi.org/10.1145/3308558.3314139>.
- [35] Manfredi Napolitano et al. “FlowTwin: A Digital Twin for Traffic Flow Monitoring”. In: *IEEE Open Journal of Intelligent Transportation Systems* 6 (2025), pp. 1551–1568. DOI: 10.1109/OJITS.2025.3637341.
- [36] Federal Highway Administration. *Traffic Analysis Toolbox Volume III: Guidelines for Applying Traffic Microsimulation Modeling Software 2019 Update to the 2004 Version – Chapter 6. Alternatives Analysis*. 2019. URL: <https://ops.fhwa.dot.gov/publications/fhwahop18036/chapter6.htm> (visited on 05/07/2026).
- [37] Averill M. Law and W. David Kelton. *Simulation Modeling and Analysis*. 3rd. New York: McGraw-Hill, 2000.
- [38] Wilco Burghout. “A note on the number of replication runs in stochastic traffic simulation models”. In: (Jan. 2004).
- [39] Texas Department of Transportation. *Traffic and Safety Analysis Procedures Manual: 13.7.2 Number of Simulation Runs*. 2026. URL: <https://www.txdot.gov/manuals/des/tsp/chapter-13-microsimulation-analysis/13-7-alternatives-analysis/13-7-2-number-of-simulation-runs.html> (visited on 05/07/2026).
- [40] Seong-Hee Kim and Barry L. Nelson. “Selecting the Best System”. In: *Simulation*. Vol. 13. Handbooks in Operations Research and Management Science. Elsevier, 2006, pp. 501–534. DOI: 10.1016/S0927-0507(06)13017-0.

- [41] Chun-Hung Chen et al. “Simulation Budget Allocation for Further Enhancing the Efficiency of Ordinal Optimization”. In: *Discrete Event Dynamic Systems* 10.3 (2000), pp. 251–270. DOI: 10.1023/A:1008349927281.
- [42] Zaile Li, Weiwei Fan, and L. Jeff Hong. *The (Surprising) Sample Optimality of Greedy Procedures for Large-Scale Ranking and Selection*. 2023. arXiv: 2303.02951. URL: <https://arxiv.org/abs/2303.02951>.
- [43] J. S. Dodgson et al. *Multi-criteria analysis: a manual*. Tech. rep. London: Department for Communities and Local Government, 2009. URL: https://eprints.lse.ac.uk/12761/1/Multi-criteria_Analysis.pdf (visited on 05/07/2026).
- [44] Donald E. Grierson. “Pareto multi-criteria decision making”. In: *Advanced Engineering Informatics* 22.3 (2008). Collaborative Design and Manufacturing, pp. 371–384. ISSN: 1474-0346. DOI: <https://doi.org/10.1016/j.aei.2008.03.001>. URL: <https://www.sciencedirect.com/science/article/pii/S1474034608000281>.
- [45] Evangelos Triantaphyllou and Alfonso Sánchez. “A Sensitivity Analysis Approach for Some Deterministic Multi-Criteria Decision-Making Methods*”. In: *Decision Sciences* 28 (Jan. 1997), pp. 151–194. DOI: 10.1111/j.1540-5915.1997.tb01306.x.
- [46] Stan Development Team. *Stan Functions Reference: Simplex Distributions*. 2026. URL: https://mc-stan.org/docs/functions-reference/simplex_distributions.html (visited on 05/07/2026).
- [47] A.G. Wilson. “A statistical theory of spatial distribution models”. In: *Transportation Research* 1.3 (1967), pp. 253–269. ISSN: 0041-1647. DOI: [https://doi.org/10.1016/0041-1647\(67\)90035-4](https://doi.org/10.1016/0041-1647(67)90035-4). URL: <https://www.sciencedirect.com/science/article/pii/0041164767900354>.
- [48] Jin Y. Yen. “Finding the K Shortest Loopless Paths in a Network”. In: *Management Science* 17.11 (1971), pp. 712–716. ISSN: 00251909, 15265501. URL: <http://www.jstor.org/stable/2629312> (visited on 04/16/2026).
- [49] Transportation Research Board. *Highway Capacity Manual*. National Academies of Sciences, Engineering, and Medicine. Washington, D.C., 2000.
- [50] German Aerospace Center (DLR) and others. *Rerouter*. 2026. URL: <https://sumo.dlr.de/docs/Simulation/Rerouter.html> (visited on 07/01/2026).
- [51] Xavier Ponsard. *Chantier à Bruxelles : le tunnel Delta sera fermé à partir de ce 15 juillet*. RTBF. July 2024. URL: <https://www.rtb.be/article/chantier-a-bruxelles-le-tunnel-delta-sera-ferme-a-partir-de-ce-15-juillet-11403826> (visited on 08/07/2026).